Multiclass Classification

01_dataset.py

```
# 01_dataset.py
# Generate a simple dataset for three-class classification.

import numpy as np
import matplotlib.pyplot as plt

# Reproducible random numbers
np.random.seed(42)

# Create three groups of data points.
class_0 = np.random.randn(100, 2) + np.array([0, 0])
class_1 = np.random.randn(100, 2) + np.array([4, 0])
class_2 = np.random.randn(100, 2) + np.array([2, 4])

# Combine the input features.
X = np.vstack((class_0, class_1, class_2))

# Create integer labels.
y = np.array(
    [0] * 100 +
    [1] * 100 +
    [2] * 100
)

# Display dataset dimensions.
print("Input shape:", X.shape)
print("Label shape:", y.shape)
print("First five samples:\n", X[:5])
print("First five labels:", y[:5])

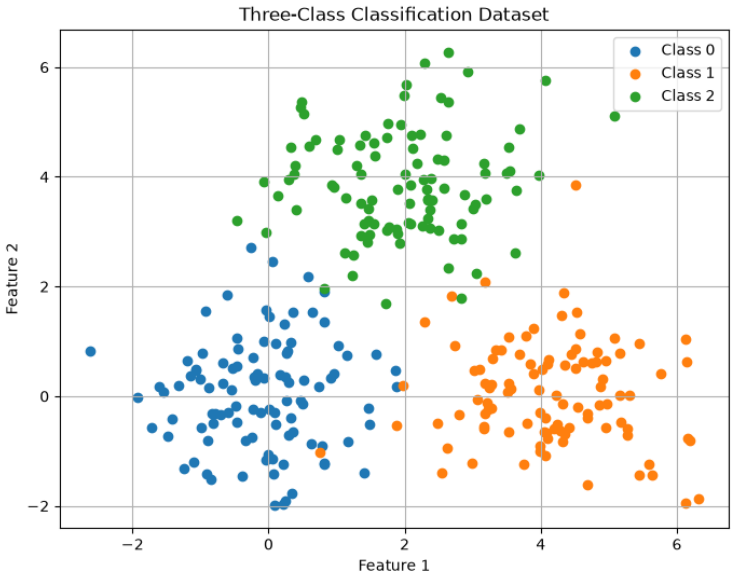
# Visualize the dataset.
plt.figure(figsize=(8, 6))

plt.scatter(class_0[:, 0], class_0[:, 1], label="Class 0")
plt.scatter(class_1[:, 0], class_1[:, 1], label="Class 1")
plt.scatter(class_2[:, 0], class_2[:, 1], label="Class 2")

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Three-Class Classification Dataset")
plt.legend()
plt.grid(True)
plt.show()
```

Code	Meaning
`np.random.randn(100, 2)`	Generates 100 samples, each having two features.
`np.array([4, 0])`	Shifts the second class to a different location.
`np.vstack(...)`	Stacks all three groups vertically into one dataset.
`[0] * 100`	Creates 100 labels belonging to class 0.
`X.shape`	Reports the number of samples and features.
`y.shape`	Reports the number of labels.

```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 01_dataset.py
Input shape: (300, 2)
Label shape: (300,)
First five samples:
 [[ 0.49671415 -0.1382643 ]
 [ 0.64768854  1.52302986]
 [-0.23415337 -0.23413696]
 [ 1.57921282  0.76743473]
 [-0.46947439  0.54256004]]
First five labels: [0 0 0 0 0]
```



This means we have 300 examples, two input features per example, and one class label per example.

02_class_labels.py

```
# 02_class_labels.py
import numpy as np

# Integer class labels for nine samples
y = np.array([0, 0, 0, 1, 1, 1, 2, 2, 2])

num_classes = 3

# Convert integer labels to one-hot encoding
y_one_hot = np.eye(num_classes)[y]

print("Integer labels:")
print(y)

print("\nOne-hot encoded labels:")
print(y_one_hot)

print("\nInteger label shape:", y.shape)
print("One-hot label shape:", y_one_hot.shape)
```

```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 02_class_labels.py
Integer labels:
[0 0 0 1 1 1 2 2 2]

One-hot encoded labels:
[[1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]]

Integer label shape: (9,)
One-hot label shape: (9, 3)
```

Two operations:

1. Operation 1: `np.eye(3)` This generates a 3 × 3 identity matrix:

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

2. Operation 2: `[y]` Python selects a row for each label. For example, if y = 2, it selects the third row:

$$y = 2 \quad \longrightarrow \quad [0,0,1]$$

   Since our dataset has nine labels, the encoded output has nine rows and three columns.

Suppose a model predicts the following probabilities for a sample whose correct label is Class 1:

$$\mathbf{p} = [0.10,\ 0.75,\ 0.15]$$

The target in one-hot form is: $\mathbf{y} = [0,\ 1,\ 0]$

The model predicts Class 1 correctly because 0.75 is the largest probability.

03_visualize_dataset.py

```
# 03_visualize_dataset.py

import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# Generate three classes
class_0 = np.random.randn(100, 2) + [0, 0]
class_1 = np.random.randn(100, 2) + [4, 0]
class_2 = np.random.randn(100, 2) + [2, 4]

# Combine all samples
X = np.vstack((class_0, class_1, class_2))

# Create class labels
y = np.array([0] * 100 + [1] * 100 + [2] * 100)

# Plot the dataset
plt.figure(figsize=(9, 7))

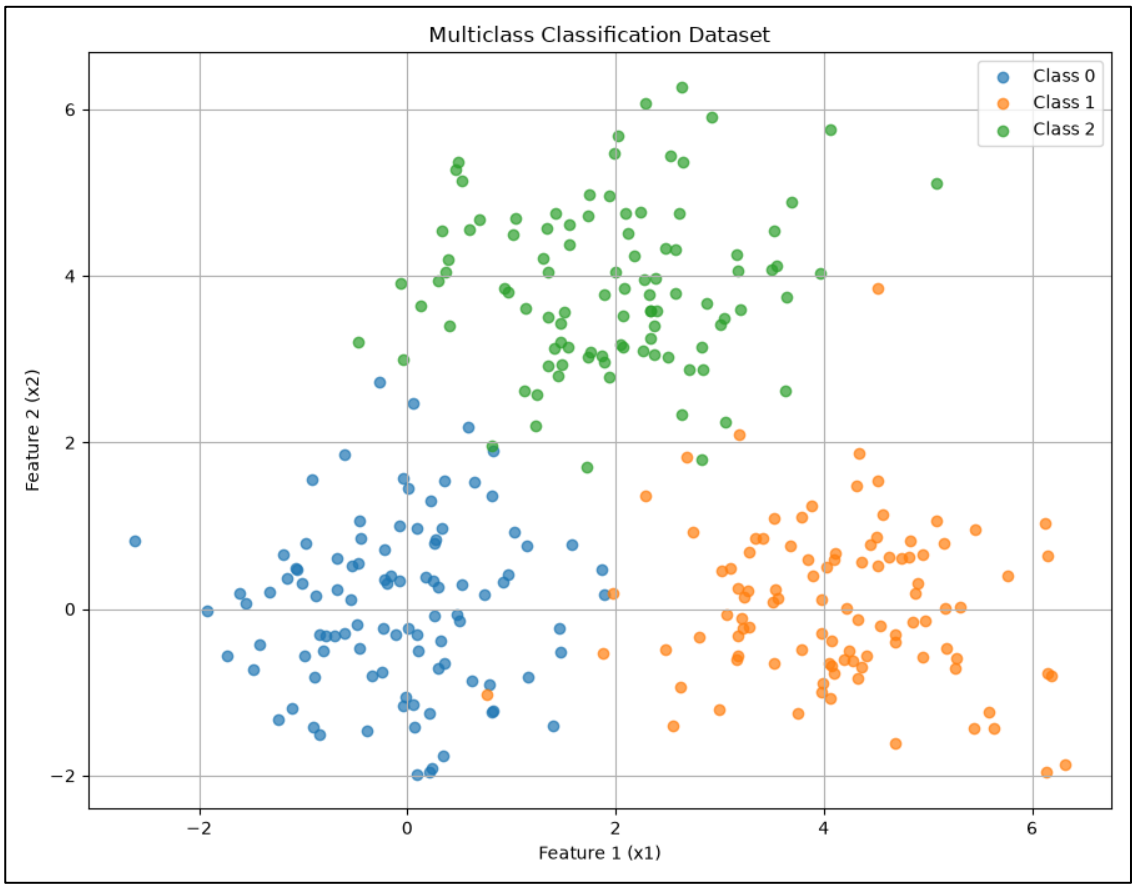
for class_id in range(3):
    mask = y == class_id
```

```
    plt.scatter(
        X[mask, 0],
        X[mask, 1],
        label=f"Class {class_id}",
        alpha=0.7
    )

print("Dataset shape:", X.shape)
print("Labels shape:", y.shape)
print("Samples per class:", np.bincount(y))
print("Saved plot as 03_dataset.png")

plt.xlabel("Feature 1 (x1)")
plt.ylabel("Feature 2 (x2)")
plt.title("Multiclass Classification Dataset")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("03_dataset.png", dpi=150)
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 03_visualize_dataset.py
Dataset shape: (300, 2)
Labels shape: (300,)
Samples per class: [100 100 100]
Saved plot as 03_dataset.png
```



Logits-

A logit is the raw score a model produces for a class before applying softmax.

In binary classification, we learned the linear expression: $z = wx + b$

Here:

- x is an input feature.
- w is a learned weight.
- b is a bias.
- z is the raw score, or logit.

In multiclass classification, we need one score for each class.

Suppose our model classifies flowers into three classes:

- Class 0: Setosa
- Class 1: Versicolor
- Class 2: Virginica

The model produces three logits: $\mathbf{z} = [z_0, z_1, z_2]$

For example: $\mathbf{z} = [2.0,\ 1.0,\ 0.1]$

These are scores, not probabilities. They don't have to lie between 0 and 1, and they don't have to sum to 1.

Our dataset has two features, x_1 and x_2. We want three output scores.

Each class gets its own pair of weights and its own bias.

$$z_0 = w_{10}x_1 + w_{20}x_2 + b_0$$
$$z_1 = w_{11}x_1 + w_{21}x_2 + b_1$$
$$z_2 = w_{12}x_1 + w_{22}x_2 + b_2$$

Let's choose an illustrative input: $\mathbf{x} = [2,3]$

And these example parameters:

$$W = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 1 \end{bmatrix}, \qquad \mathbf{b} = [0,\ 0.5,\ -0.5]$$

For Class 0: $z_0 = (1)(2) + (0)(3) + 0 = 2$

For Class 1: $z_1 = (0)(2) + (1)(3) + 0.5 = 3.5$

For Class 2: $z_2 = (-1)(2) + (1)(3) - 0.5 = 0.5$

Therefore, the model's output is: $\boxed{\mathbf{z} = [2,\ 3.5,\ 0.5]}$

Class 1 has the highest score. We can already predict Class 1 by choosing the largest logit, even before calculating probabilities.

Instead of calculating each class separately, we can calculate all three logits at once:

$$\boxed{\mathbf{z} = \mathbf{x}W + \mathbf{b}}$$

1 × 2: Input X, 2 features

2 × 3: Weights W, 3 classes

1 × 3: Logits Z, 3 scores

The weight matrix is: $W = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 1 \end{bmatrix}$

Each column contains the weights for one class. Each row corresponds to one input feature.

04_logits.py

```
# 04_logits.py
# Calculate logits for a three-class classifier.

import numpy as np

# One sample with two input features
X = np.array([2.0, 3.0])

# Weight matrix: 2 features x 3 classes
W = np.array([
```

```
    [1.0,  0.0, -1.0],
    [0.0,  1.0,  1.0]
])

# One bias per class
b = np.array([0.0, 0.5, -0.5])

# Calculate all three logits
logits = X @ W + b

print("Input features:", X)
print("Weight matrix:\n", W)
print("Bias vector:", b)
print("Logits:", logits)

# Find the class with the highest score
predicted_class = np.argmax(logits)

print("Predicted class:", predicted_class)
```

```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 04_logits.py
Input features: [2. 3.]
Weight matrix:
 [[ 1.  0. -1.]
 [ 0.  1.  1.]]
Bias vector: [ 0.   0.5 -0.5]
Logits: [2.  3.5 0.5]
Predicted class: 1
```

Softmax-

For K classes, softmax calculates the probability of class k as: $\boxed{p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}}$

- z_k: The logit for class k.
- e^{z_k}: The exponential of that logit.
- $\sum_j e^{z_j}$: The sum of the exponentials of all class logits.
- p_k: The resulting probability for that class.

The exponential makes every score positive. Dividing each exponential by their total makes all probabilities sum to 1.

For $\mathbf{z} = [2,\ 3.5,\ 0.5]$

$$e^2 \approx 7.3891$$
$$e^{3.5} \approx 33.1155$$
$$e^{0.5} \approx 1.6487$$
$$S = 7.3891 + 33.1155 + 1.6487$$
$$S \approx 42.1533$$

For Class 0: $p_0 = \frac{7.3891}{42.1533} \approx 0.1753$

For Class 1: $p_1 = \frac{33.1155}{42.1533} \approx 0.7856$

For Class 2: $p_2 = \frac{1.6487}{42.1533} \approx 0.0391$

Therefore, predicted probabilities:
Class 0: **17.53%**
Class 1: **78.56%**
Class 2: **3.91%**
Probability total: **100.00%**

Our predicted class is Class 1, with a probability of approximately 0.7856.
Notice that softmax preserves the ranking of the logits: the class with the largest logit also has the largest softmax probability.

05_softmax.py

```
# 05_softmax.py
# Implement softmax from scratch using NumPy.

import numpy as np

# Logits calculated in Lesson 4
logits = np.array([2.0, 3.5, 0.5])

# Step 1: Subtract the largest logit for numerical stability
shifted_logits = logits - np.max(logits)

# Step 2: Calculate exponentials
exp_values = np.exp(shifted_logits)

# Step 3: Calculate the sum of exponentials
total = np.sum(exp_values)

# Step 4: Normalize to obtain probabilities
probabilities = exp_values / total

# Step 5: Predict the class with the highest probability
predicted_class = np.argmax(probabilities)

print("Original logits:", logits)
print("Shifted logits:", shifted_logits)
print("Exponentials:", exp_values)
print("Sum of exponentials:", total)
print("Probabilities:", probabilities)
print("Probability sum:", np.sum(probabilities))
print("Predicted class:", predicted_class)
```

```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 05_softmax.py
Original logits: [2.  3.5 0.5]
Shifted logits: [-1.5  0.  -3. ]
Exponentials: [0.22313016 1.         0.04978707]
Sum of exponentials: 1.2729172285162937
Probabilities: [0.17529039 0.78559703 0.03911257]
Probability sum: 1.0
Predicted class: 1
```

Categorical Cross-Entropy Loss-
Cross-entropy loss measures how much probability the model assigns to the correct answer.
For one sample with K classes:

$$L = -\sum_{k=1}^{K} y_k \log(p_k)$$

- L: loss for one sample.
- y_k: target value for class k, usually 0 or 1 in one-hot encoding.
- p_k: predicted probability for class k.
- log: natural logarithm.

Because only the correct class has target value 1, all other terms vanish.

For our target [0,1,0], the equation simplifies to: $\boxed{L = -\log(p_{\text{correct}})}$

Since the correct class is Class 1 and its predicted probability is 0.70: $L = -\log(0.70) \rightarrow \boxed{L \approx 0.3567}$
A smaller loss means the model assigned more probability to the correct answer.

Imagine three models predict the same sample. The correct answer is Class 1.

Model A — Good prediction, Low loss Probabilities: [0.10, 0.90, 0.00] Loss: $-\log(0.90) \approx 0.1054$
Model B — Uncertain prediction, Moderate loss Probabilities: [0.20, 0.50, 0.30] Loss: $-\log(0.50) \approx 0.6931$
Model C — Wrong prediction, High loss Probabilities: [0.90, 0.05, 0.05] Loss: $-\log(0.05) \approx 2.9957$

- Model A gives 90% probability to the correct class, so its loss is small.
- Model B gives only 50%, so its loss is higher.
- Model C gives only 5% to the correct class, so its loss is much higher.

This is why cross-entropy is useful for training: it rewards assigning high probability to the true class and strongly penalizes assigning it very little probability.

Real training datasets contain many samples, not just one.
Suppose we have three samples with the following targets and predictions.

Sample	**Correct class**	**Predicted probability of correct class**	**Loss**
1	0	0.80	$-\log(0.80) = 0.2231$
2	1	0.60	$-\log(0.60) = 0.5108$
3	2	0.90	$-\log(0.90) = 0.1054$

The mean batch loss is:

$$L_{\text{mean}} = \frac{L_1 + L_2 + L_3}{3}$$

$$L_{\text{mean}} = \frac{0.2231 + 0.5108 + 0.1054}{3}$$

$$\boxed{L_{\text{mean}} \approx 0.2798}$$

We usually minimize this mean loss during training. It gives one value representing the model's performance across the batch.

06_cross_entropy.py

```
# 06_cross_entropy.py
# Calculate categorical cross-entropy from scratch.

import numpy as np

# Three samples, three classes each
probabilities = np.array([
    [0.80, 0.10, 0.10],
    [0.20, 0.60, 0.20],
    [0.05, 0.05, 0.90]
])

# Correct class for each sample
y = np.array([0, 1, 2])
```

```
# Select the probability assigned to each correct class
correct_probs = probabilities[np.arange(len(y)), y]

# Avoid log(0) in more general cases
epsilon = 1e-15
correct_probs = np.clip(correct_probs, epsilon, 1.0)

# Calculate loss for each sample
losses = -np.log(correct_probs)

# Calculate average loss
mean_loss = np.mean(losses)

print("Correct class probabilities:", correct_probs)
print("Individual losses:", losses)
print("Mean cross-entropy loss:", mean_loss)
```

```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 06_cross_entropy.py
Correct class probabilities: [0.8 0.6 0.9]
Individual losses: [0.22314355 0.51082562 0.10536052]
Mean cross-entropy loss: 0.2797765635793423
```

Softmax + Cross-Entropy-
Suppose our model produces these logits: $\mathbf{z} = [2,\ 3.5,\ 0.5]$
The softmax probabilities are approximately: $\mathbf{p} = [0.1753,\ 0.7856,\ 0.0391]$
The predicted class is Class 1 because 0.7856 is the highest probability.

Now suppose the true label is also Class 1.
Its one-hot target is: $\mathbf{y} = [0,\ 1,\ 0]$
The cross-entropy loss is: $L = -\sum_{k=0}^{2} y_k \log(p_k)$
Substituting the target: $L = -[0\log(0.1753) + 1\log(0.7856) + 0\log(0.0391)] \rightarrow \boxed{L \approx 0.2412}$
The model predicts the correct class, but the loss is not zero because it assigns only about 78.56% probability to that class.

Now imagine the true class were Class 0 instead. The same probabilities would produce: $L = -\log(0.1753) \approx 1.7415$
The prediction would be wrong, and the loss would be much larger.

07_loss_calculation.py

```
# 07_loss_calculation.py
# Complete multiclass classification pipeline using NumPy.

import numpy as np

# Three samples, each with two input features
X = np.array([
    [2.0, 3.0],
    [1.0, 2.0],
    [3.0, 1.0]
])

# Weight matrix: 2 features x 3 classes
W = np.array([
    [1.0,  0.0, -1.0],
    [0.0,  1.0,  1.0]
])

# One bias for each class
b = np.array([0.0, 0.5, -0.5])
```

```
# Correct integer labels for the three samples
y = np.array([1, 2, 0])

# STEP 1: Calculate logits for every sample
logits = X @ W + b

# STEP 2: Calculate numerically stable softmax
shifted_logits = logits - np.max(
    logits, axis=1, keepdims=True
)

exp_values = np.exp(shifted_logits)

probabilities = exp_values / np.sum(
    exp_values, axis=1, keepdims=True
)

# STEP 3: Select the probability of each true class
correct_probs = probabilities[np.arange(len(y)), y]

# STEP 4: Calculate cross-entropy for each sample
epsilon = 1e-15
losses = -np.log(np.clip(correct_probs, epsilon, 1.0))

# STEP 5: Calculate average loss
mean_loss = np.mean(losses)

# STEP 6: Predict the class with the highest probability
predictions = np.argmax(probabilities, axis=1)

# STEP 7: Calculate accuracy
accuracy = np.mean(predictions == y)

print("Logits:\n", np.round(logits, 4))
print("\nProbabilities:\n", np.round(probabilities, 4))
print("\nTrue labels:", y)
print("Predictions:", predictions)
print("Correct-class probabilities:", np.round(correct_probs, 4))
print("Individual losses:", np.round(losses, 4))
print("Mean loss:", round(mean_loss, 4))
print("Accuracy:", round(accuracy * 100, 2), "%")
```

```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 07_loss_calculation.py
Logits:
 [[ 2.   3.5  0.5]
 [ 1.   2.5  0.5]
 [ 3.   1.5 -2.5]]

Probabilities:
 [[0.1753 0.7856 0.0391]
 [0.1643 0.7361 0.0996]
 [0.8149 0.1818 0.0033]]

True labels: [1 2 0]
Predictions: [1 1 0]
Correct-class probabilities: [0.7856 0.0996 0.8149]
Individual losses: [0.2413 2.3064 0.2047]
Mean loss: 0.9175
Accuracy: 66.67 %
```

Gradients in Multiclass Classification-

A gradient tells us how sensitive the loss is to a change in a parameter.

For a weight w, the gradient is: $\frac{\partial L}{\partial w}$

- If the gradient is positive, increasing that weight slightly tends to increase the loss.

- If the gradient is negative, increasing that weight slightly tends to decrease the loss.
- If its magnitude is large, the loss is more sensitive to a small change in that weight.

Gradient descent uses this information to adjust the parameters in a direction that reduces the loss.

For multiclass classification:

Linear layer	$z_k = \sum_i x_i w_{ik} + b_k$
Softmax	$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$
Cross-entropy	$L = -\sum_k y_k \log p_k$
Logit Gradient	$\boxed{\frac{\partial L}{\partial z_k} = p_k - y_k}$

Derivation:

For one sample: $L = -\sum_k y_k \log(p_k)$

Softmax: $p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$

The derivative of a softmax output p_j with respect to a logit z_k is: $\frac{\partial p_j}{\partial z_k} = p_j(\delta_{jk} - p_k)$

Here, $\delta_{jk} = 1$ when $j = k$, and 0 otherwise.

This derivative accounts for the fact that changing one logit affects all the softmax probabilities because they share a denominator.

Differentiate the loss with respect to z_k: $\frac{\partial L}{\partial z_k} = -\sum_j \frac{y_j}{p_j}\frac{\partial p_j}{\partial z_k} = -\sum_j y_j\left(\delta_{jk} - p_k\right) = -y_k + p_k \sum_j y_j$

For a one-hot target, the target values sum to 1: $\sum_j y_j = 1$

Therefore: $\boxed{\frac{\partial L}{\partial z_k} = p_k - y_k}$

This formula works for every class in a single-label multiclass problem.

Since, $z_k = \sum_i x_i w_{ik} + b_k$

Using the chain rule:

$$\boxed{\frac{\partial L}{\partial w_{ik}} = x_i(p_k - y_k)}$$

$$\boxed{\frac{\partial L}{\partial b_k} = p_k - y_k}$$

Therefore:

$$\boxed{\frac{\partial L}{\partial z_k} = p_k - y_k}$$

$$\boxed{\frac{\partial L}{\partial w_{ik}} = x_i(p_k - y_k)}$$

$$\boxed{\frac{\partial L}{\partial b_k} = p_k - y_k}$$

Example:

Let's use an input sample: $\mathbf{x} = [2, -1]$

Its logit gradient is: $\mathbf{g} = [0.20, -0.50, 0.30]$

The weight gradient is the outer product of the input and the logit gradient:

$$\frac{\partial L}{\partial W} = \mathbf{x}^T \mathbf{g}$$

$$\boxed{\frac{\partial L}{\partial W} = \begin{bmatrix} 0.40 & -1.00 & 0.60 \\ -0.20 & 0.50 & -0.30 \end{bmatrix}}$$

The bias gradient is simply:

$$\boxed{\frac{\partial L}{\partial \mathbf{b}} = [0.20, -0.50, 0.30]}$$

08_gradients.py

```
# 08_gradients.py
# Calculate gradients for softmax + cross-entropy.

import numpy as np

# One sample, two input features
X = np.array([2.0, -1.0])

# Three predicted probabilities
probabilities = np.array([0.20, 0.50, 0.30])

# Correct class is Class 1
y = np.array([0.0, 1.0, 0.0])

# Step 1: Gradient of loss with respect to logits
dz = probabilities - y

# Step 2: Gradient of loss with respect to weights
dW = np.outer(X, dz)

# Step 3: Gradient of loss with respect to biases
db = dz.copy()

print("Predicted probabilities:", probabilities)
print("True one-hot label:", y)

print("\nGradient with respect to logits:")
print(dz)

print("\nGradient with respect to weights:")
print(dW)

print("\nGradient with respect to biases:")
print(db)

print("\nSum of logit gradients:", np.sum(dz))
```

```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 08_gradients.py
Predicted probabilities: [0.2 0.5 0.3]
True one-hot label: [0. 1. 0.]

Gradient with respect to logits:
[ 0.2 -0.5  0.3]

Gradient with respect to weights:
[[ 0.4 -1.   0.6]
 [-0.2  0.5 -0.3]]

Gradient with respect to biases:
[ 0.2 -0.5  0.3]

Sum of logit gradients: 0.0
```

Gradient Descent-

The gradient tells us how the loss changes with the logits. Gradient descent uses these gradients to adjust the parameters in the direction that locally reduces the loss.

The update rule is:

$$\boxed{W_{\text{new}} = W_{\text{old}} - \eta\, dW}$$

$$\boxed{b_{\text{new}} = b_{\text{old}} - \eta\, db}$$

- W: weight matrix.
- b: bias vector.
- dW: gradient of loss with respect to weights.
- db: gradient of loss with respect to biases.
- η: learning rate.

Example:

Our input sample is: $\mathbf{x} = [2, -1]$

The true class is Class 1: $\mathbf{y} = [0,1,0]$

The initial parameters are: $W = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \qquad b = [0,0,0]$

The initial logits: $\mathbf{z} = \mathbf{x}W + b = [0,0,0]$

Because all logits are equal, softmax produces equal probabilities: $\mathbf{p} = [1/3,1/3,1/3]$

The initial loss is: $L = -\log(1/3) = \log 3 \rightarrow \boxed{L \approx 1.0986}$

The logit gradient is: $dz = \mathbf{p} - \mathbf{y} = [1/3, -2/3,1/3]$

The weight gradient is: $dW = \mathbf{x}^T dz = \begin{bmatrix} 2/3 & -4/3 & 2/3 \\ -1/3 & 2/3 & -1/3 \end{bmatrix}$

The bias gradient is: $db = [1/3, -2/3,1/3]$

Choose a learning rate of: $\eta = 0.1$

Apply gradient descent: $W_{\text{new}} = W - 0.1dW \approx \begin{bmatrix} -0.0667 & 0.1333 & -0.0667 \\ 0.0333 & -0.0667 & 0.0333 \end{bmatrix}$

Similarly: $b_{\text{new}} \approx [-0.0333, 0.0667, -0.0333]$

Notice that the parameters for Class 1 have moved in the direction that tends to increase its score relative to the other classes.

Using the updated parameters, the new logits are: $\mathbf{z}_{\text{new}} = [-0.2, 0.4, -0.2]$

The probability of the correct class becomes approximately 0.4767.

The new loss is: $L_{\text{new}} = -\log(0.4767) \rightarrow \boxed{L_{\text{new}} \approx 0.7410}$

09_gradient_descent.py

```
# 09_gradient_descent.py
# Perform one gradient descent update for multiclass classification.

import numpy as np

# One training sample with two features
X = np.array([2.0, -1.0])

# True class: Class 1
y = np.array([0.0, 1.0, 0.0])

# Start with zero weights and biases
W = np.zeros((2, 3))
b = np.zeros(3)

# Learning rate
learning_rate = 0.1


def softmax(logits):
    # Stable softmax
```

```
    shifted = logits - np.max(logits)
    exp_values = np.exp(shifted)
    return exp_values / np.sum(exp_values)


def cross_entropy(probabilities, target):
    # Loss for a one-hot target
    correct_probability = np.sum(probabilities * target)
    return -np.log(max(correct_probability, 1e-15))


# ----- BEFORE UPDATE -----

logits_before = X @ W + b
probs_before = softmax(logits_before)
loss_before = cross_entropy(probs_before, y)

# ----- CALCULATE GRADIENTS -----

# Gradient of loss with respect to logits
dz = probs_before - y

# Gradients of weights and biases
dW = np.outer(X, dz)
db = dz.copy()

# ----- UPDATE PARAMETERS -----

W = W - learning_rate * dW
b = b - learning_rate * db

# ----- AFTER UPDATE -----

logits_after = X @ W + b
probs_after = softmax(logits_after)
loss_after = cross_entropy(probs_after, y)

print("Initial probabilities:", probs_before)
print("Initial loss:", round(loss_before, 4))

print("\nWeight gradient:\n", dW)
print("Bias gradient:", db)

print("\nUpdated weights:\n", W)
print("Updated biases:", b)

print("\nNew logits:", logits_after)
print("New probabilities:", probs_after)
print("New loss:", round(loss_after, 4))
print("Loss decreased:", loss_after < loss_before)
```

```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 09_gradient_descent.py
Initial probabilities: [0.33333333 0.33333333 0.33333333]
Initial loss: 1.0986

Weight gradient:
 [[ 0.66666667 -1.33333333  0.66666667]
 [-0.33333333  0.66666667 -0.33333333]]
Bias gradient: [ 0.33333333 -0.66666667  0.33333333]

Updated weights:
 [[-0.06666667  0.13333333 -0.06666667]
 [ 0.03333333 -0.06666667  0.03333333]]
Updated biases: [-0.03333333  0.06666667 -0.03333333]

New logits: [-0.2  0.4 -0.2]
New probabilities: [0.26163499 0.47673003 0.26163499]
New loss: 0.7408
Loss decreased: True
```

Training-
Our model has two input features and three output classes.
The model calculates: $Z = XW + b$
Then softmax converts logits into probabilities: $P = \text{softmax}(Z)$
Cross-entropy measures the prediction error: $L = -\frac{1}{N}\sum_{i=1}^{N} \log p_{i,y_i}$
Here, N is the number of samples, and p_{i,y_i} is the probability assigned to the correct class for sample i.
We calculate gradients and update the parameters:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

$$b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

We repeat this process until the model learns useful decision boundaries or reaches our chosen training limit.

An **epoch** is one complete pass through the training dataset. Our dataset contains 300 samples, with 100 samples per class. Each epoch calculates the loss and updates the model parameters once using the full dataset.

10_training.py

```
# 10_training.py
# Train a multiclass classifier from scratch using NumPy.

import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# ------------------------------------------------
# STEP 1: GENERATE THE THREE-CLASS DATASET
# ------------------------------------------------

class_0 = np.random.randn(100, 2) + [0, 0]
class_1 = np.random.randn(100, 2) + [4, 0]
class_2 = np.random.randn(100, 2) + [2, 4]

X = np.vstack((class_0, class_1, class_2))

y = np.array(
    [0] * 100 +
    [1] * 100 +
    [2] * 100
)

# Shuffle the samples before splitting
indices = np.random.permutation(len(y))
X = X[indices]
y = y[indices]

# 80% training data, 20% test data
split = int(0.8 * len(y))

X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

print("Training samples:", len(y_train))
print("Test samples:", len(y_test))

# ------------------------------------------------
# STEP 2: INITIALIZE MODEL PARAMETERS
# ------------------------------------------------

num_features = 2
num_classes = 3
```

```
W = np.random.randn(num_features, num_classes) * 0.01
b = np.zeros(num_classes)

learning_rate = 0.1
epochs = 500

loss_history = []

# ------------------------------------------------
# STEP 3: DEFINE SOFTMAX
# ------------------------------------------------

def softmax(logits):
    shifted = logits - np.max(
        logits, axis=1, keepdims=True
    )

    exp_values = np.exp(shifted)

    return exp_values / np.sum(
        exp_values, axis=1, keepdims=True
    )

# ------------------------------------------------
# STEP 4: TRAIN THE MODEL
# ------------------------------------------------

N = len(y_train)
rows = np.arange(N)

for epoch in range(epochs):

    # Forward pass: calculate logits and probabilities
    logits = X_train @ W + b
    probabilities = softmax(logits)

    # Calculate average cross-entropy loss
    correct_probs = probabilities[rows, y_train]
    loss = -np.mean(np.log(np.clip(
        correct_probs, 1e-15, 1.0
    )))

    loss_history.append(loss)

    # Gradient of loss with respect to logits
    dZ = probabilities.copy()
    dZ[rows, y_train] -= 1
    dZ /= N

    # Gradients for weights and biases
    dW = X_train.T @ dZ
    db = np.sum(dZ, axis=0)

    # Gradient descent update
    W -= learning_rate * dW
    b -= learning_rate * db

    if (epoch + 1) % 100 == 0:
        print(
            f"Epoch {epoch + 1:3d} | "
            f"Loss: {loss:.4f}"
        )

# ------------------------------------------------
# STEP 5: EVALUATE ON UNSEEN TEST DATA
# ------------------------------------------------

test_logits = X_test @ W + b
test_probabilities = softmax(test_logits)
```

```
test_predictions = np.argmax(
    test_probabilities, axis=1
)

test_accuracy = np.mean(
    test_predictions == y_test
)

print("\nFinal training loss:", round(loss_history[-1], 4))
print("Test accuracy:", round(test_accuracy * 100, 2), "%")

# ------------------------------------------------
# STEP 6: PLOT THE LEARNING CURVE
# ------------------------------------------------

plt.figure(figsize=(8, 5))

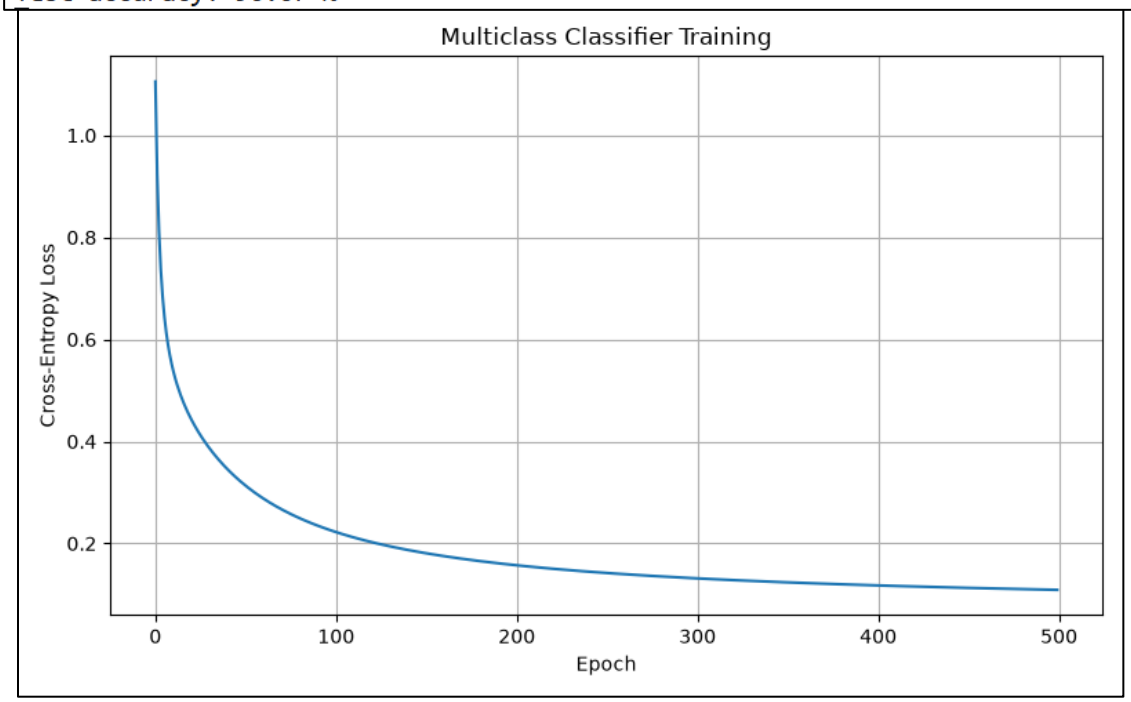
plt.plot(loss_history)

plt.xlabel("Epoch")
plt.ylabel("Cross-Entropy Loss")
plt.title("Multiclass Classifier Training")
plt.grid(True)
plt.tight_layout()
plt.savefig("10_training_loss.png", dpi=150)
plt.show()

print("Saved plot as 10_training_loss.png")
```

```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 10_training.py
Training samples: 240
Test samples: 60
Epoch 100 | Loss: 0.2234
Epoch 200 | Loss: 0.1575
Epoch 300 | Loss: 0.1317
Epoch 400 | Loss: 0.1178
Epoch 500 | Loss: 0.1091

Final training loss: 0.1091
Test accuracy: 96.67 %
```



Decision boundary-

A decision boundary is where two or more classes have equal scores and the model's preferred class changes.
Suppose the three class scores are:

$$z_0 = w_{10}x_1 + w_{20}x_2 + b_0$$
$$z_1 = w_{11}x_1 + w_{21}x_2 + b_1$$
$$z_2 = w_{12}x_1 + w_{22}x_2 + b_2$$
$$\mathbf{z} = \mathbf{x}W + \mathbf{b}$$

The model predicts whichever class has the highest score.
For example, the boundary between Class 0 and Class 1 occurs where: $z_0 = z_1$
Substituting the two linear equations gives: $(w_{10} - w_{11})x_1 + (w_{20} - w_{21})x_2 + (b_0 - b_1) = 0$
This is the equation of a straight line.
Softmax does not change which class has the highest score, so the decision boundaries can be calculated directly from the logits.

11_visualize_learning.py

```
# 11_visualize_learning.py
# Train a three-class classifier and visualize its decision boundaries.

import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# ------------------------------------------------
# STEP 1: GENERATE THE DATASET
# ------------------------------------------------

class_0 = np.random.randn(100, 2) + [0, 0]
class_1 = np.random.randn(100, 2) + [4, 0]
class_2 = np.random.randn(100, 2) + [2, 4]

X = np.vstack((class_0, class_1, class_2))
y = np.array([0] * 100 + [1] * 100 + [2] * 100)

# Shuffle and split
indices = np.random.permutation(len(y))
X = X[indices]
y = y[indices]

split = int(0.8 * len(y))

X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# ------------------------------------------------
# STEP 2: INITIALIZE THE MODEL
# ------------------------------------------------

num_features = 2
num_classes = 3

W = np.random.randn(num_features, num_classes) * 0.01
b = np.zeros(num_classes)

learning_rate = 0.1
epochs = 500
N = len(y_train)
rows = np.arange(N)


def softmax(logits):
```

```
    shifted = logits - np.max(
        logits, axis=1, keepdims=True
    )
    exp_values = np.exp(shifted)
    return exp_values / np.sum(
        exp_values, axis=1, keepdims=True
    )


# ------------------------------------------------
# STEP 3: TRAIN THE CLASSIFIER
# ------------------------------------------------

loss_history = []

for epoch in range(epochs):
    logits = X_train @ W + b
    probabilities = softmax(logits)

    correct_probs = probabilities[rows, y_train]
    loss = -np.mean(
        np.log(np.clip(correct_probs, 1e-15, 1.0))
    )
    loss_history.append(loss)

    dZ = probabilities.copy()
    dZ[rows, y_train] -= 1
    dZ /= N

    dW = X_train.T @ dZ
    db = np.sum(dZ, axis=0)

    W -= learning_rate * dW
    b -= learning_rate * db

# ------------------------------------------------
# STEP 4: EVALUATE THE TEST DATA
# ------------------------------------------------

test_probabilities = softmax(X_test @ W + b)
test_predictions = np.argmax(test_probabilities, axis=1)

accuracy = np.mean(test_predictions == y_test)

print("Learned weights:\n", np.round(W, 3))
print("Learned biases:", np.round(b, 3))
print("Final recorded training loss:", round(loss_history[-1], 4))
print("Test accuracy:", round(accuracy * 100, 2), "%")

# ------------------------------------------------
# STEP 5: CREATE A GRID OF INPUT POINTS
# ------------------------------------------------

x1_min, x1_max = X[:, 0].min() - 1, X[:, 0].max() + 1
x2_min, x2_max = X[:, 1].min() - 1, X[:, 1].max() + 1

grid_x1, grid_x2 = np.meshgrid(
    np.linspace(x1_min, x1_max, 300),
    np.linspace(x2_min, x2_max, 300)
)

# Convert every grid point into a two-feature sample
grid_points = np.column_stack((
    grid_x1.ravel(),
    grid_x2.ravel()
))
```

```
# Predict a class for every grid point
grid_probabilities = softmax(grid_points @ W + b)
grid_predictions = np.argmax(grid_probabilities, axis=1)

# Reshape predictions to match the grid
regions = grid_predictions.reshape(grid_x1.shape)

# ------------------------------------------------
# STEP 6: PLOT DECISION REGIONS AND DATA
# ------------------------------------------------

plt.figure(figsize=(10, 7))

plt.contourf(
    grid_x1, grid_x2, regions,
    levels=[-0.5, 0.5, 1.5, 2.5],
    alpha=0.25
)

# Draw boundaries between predicted classes
plt.contour(
    grid_x1, grid_x2, regions,
    levels=[0.5, 1.5],
    linewidths=1.5
)

# Plot training samples
for class_id in range(num_classes):
    mask = y_train == class_id
    plt.scatter(
        X_train[mask, 0],
        X_train[mask, 1],
        label=f"Class {class_id} (train)",
        alpha=0.65,
        s=25
    )

# Mark misclassified test samples with black crosses
wrong = test_predictions != y_test

plt.scatter(
    X_test[wrong, 0],
    X_test[wrong, 1],
    marker="x",
    s=100,
    linewidths=2,
    label="Misclassified test samples"
)

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Learned Multiclass Decision Boundaries")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("11_decision_boundaries.png", dpi=150)
plt.show()

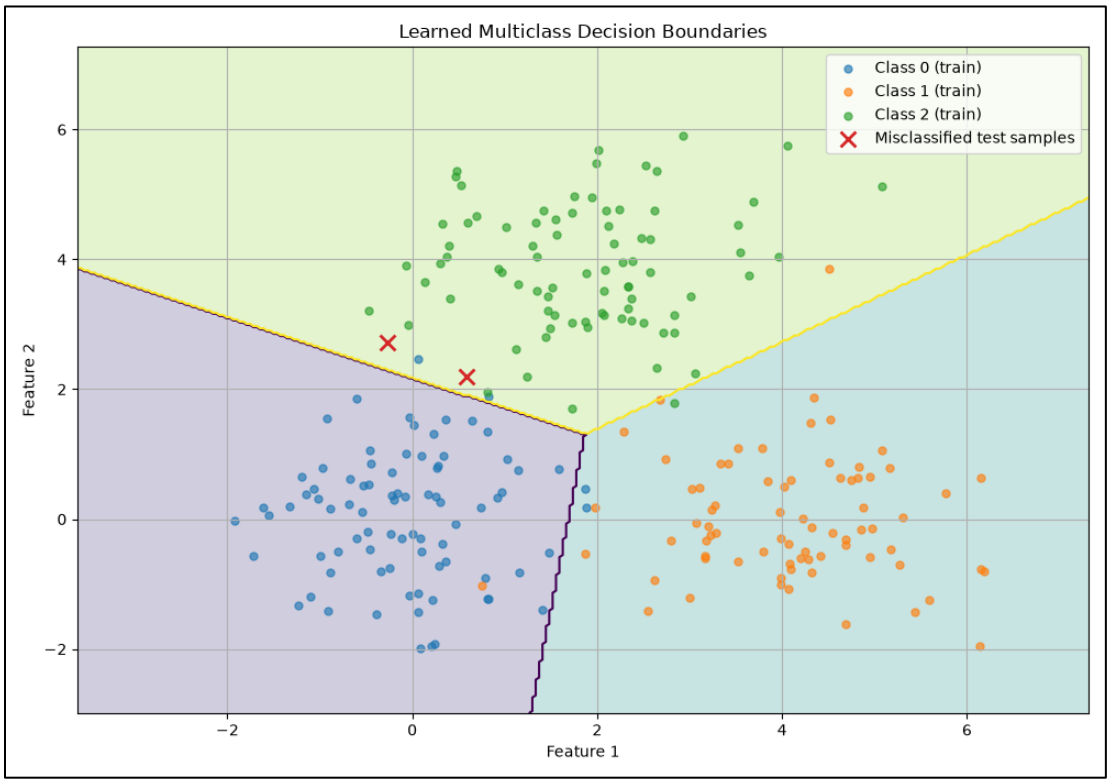
# ------------------------------------------------
# STEP 7: PLOT THE LOSS CURVE
# ------------------------------------------------

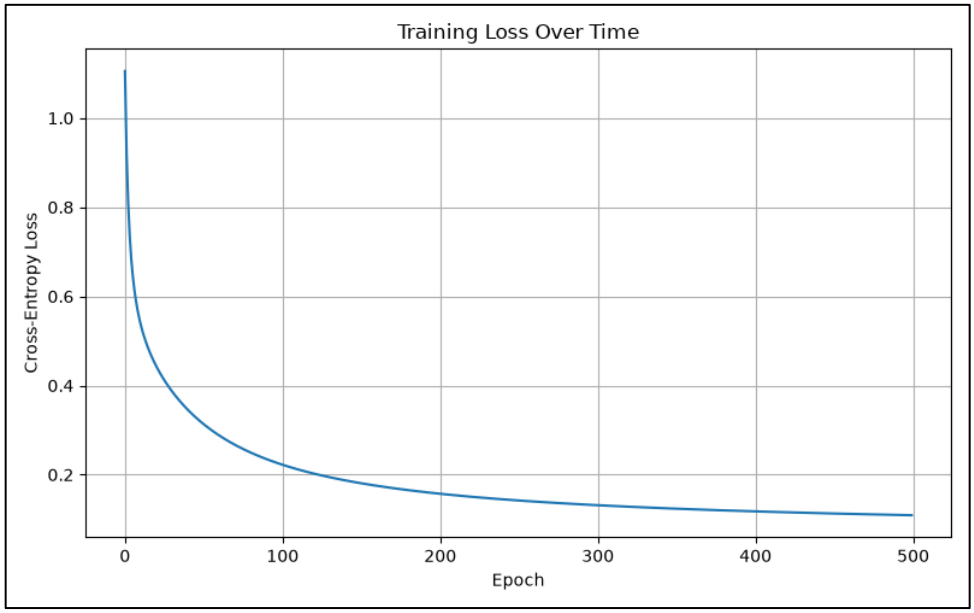
plt.figure(figsize=(8, 5))
plt.plot(loss_history)
plt.xlabel("Epoch")
```

```
plt.ylabel("Cross-Entropy Loss")
plt.title("Training Loss Over Time")
plt.grid(True)
plt.tight_layout()
plt.savefig("11_loss_curve.png", dpi=150)
plt.show()

print("Saved 11_decision_boundaries.png")
print("Saved 11_loss_curve.png")
```

```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 11_visualize_learning.py
Learned weights:
 [[-1.058  1.274 -0.188]
 [-0.504 -0.823  1.364]]
Learned biases: [ 2.666 -1.27  -1.396]
Final recorded training loss: 0.1091
Test accuracy: 96.67 %
```





12_classification_metrics.py

```
# 12_classification_metrics.py
# Multiclass evaluation from scratch using NumPy.

import numpy as np

# True labels: 10 samples from each class
y_true = np.array(
    [0] * 10 + [1] * 10 + [2] * 10
)

# Predictions constructed to match our example confusion matrix:
# Actual Class 0: 8 predicted 0, 1 predicted 1, 1 predicted 2
# Actual Class 1: 2 predicted 0, 6 predicted 1, 2 predicted 2
# Actual Class 2: 1 predicted 0, 1 predicted 1, 8 predicted 2

y_pred = np.array(
    [0] * 8 + [1] + [2] +
    [0] * 2 + [1] * 6 + [2] * 2 +
    [0] + [1] + [2] * 8
)

num_classes = 3

# STEP 1: Build confusion matrix
cm = np.zeros((num_classes, num_classes), dtype=int)

for actual, predicted in zip(y_true, y_pred):
    cm[actual, predicted] += 1

print("Confusion matrix:")
print(cm)

# STEP 2: Calculate true positives, false positives,
# and false negatives for each class.

tp = np.diag(cm)
fp = cm.sum(axis=0) - tp
fn = cm.sum(axis=1) - tp

# STEP 3: Calculate precision and recall safely
precision = np.divide(
    tp, tp + fp,
    out=np.zeros(num_classes, dtype=float),
    where=(tp + fp) != 0
)

recall = np.divide(
    tp, tp + fn,
    out=np.zeros(num_classes, dtype=float),
    where=(tp + fn) != 0
)

# STEP 4: Calculate F1-score
f1 = np.divide(
    2 * precision * recall,
    precision + recall,
    out=np.zeros(num_classes, dtype=float),
    where=(precision + recall) != 0
)

# STEP 5: Calculate overall accuracy
```

```
accuracy = np.mean(y_true == y_pred)

print("\nPer-class metrics:")
for class_id in range(num_classes):
    print(
        f"Class {class_id}: "
        f"Precision={precision[class_id]:.4f}, "
        f"Recall={recall[class_id]:.4f}, "
        f"F1={f1[class_id]:.4f}"
    )

print("\nAccuracy:", round(accuracy, 4))
print("Macro precision:", round(np.mean(precision), 4))
print("Macro recall:", round(np.mean(recall), 4))
print("Macro F1:", round(np.mean(f1), 4))
```

```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 12_classification_metrics.py
Confusion matrix:
[[8 1 1]
 [2 6 2]
 [1 1 8]]

Per-class metrics:
Class 0: Precision=0.7273, Recall=0.8000, F1=0.7619
Class 1: Precision=0.7500, Recall=0.6000, F1=0.6667
Class 2: Precision=0.7273, Recall=0.8000, F1=0.7619

Accuracy: 0.7333
Macro precision: 0.7348
Macro recall: 0.7333
Macro F1: 0.7302
```

Confusion matrix:

Actual \ Predicted	**Class 0**	**Class 1**	**Class 2**
Class 0	8	1	1
Class 1	2	6	2
Class 2	1	1	8

Class	**Precision**	**Recall**	**F1-score**
Class 0	72.73%	80.00%	76.19%
Class 1	75.00%	60.00%	66.67%
Class 2	72.73%	80.00%	76.19%

13_sklearn_multiclass.py

```
# 13_sklearn_multiclass.py
# Multiclass classification using scikit-learn.

import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
```

```
)

# ------------------------------------------------
# STEP 1: GENERATE THE DATASET
# ------------------------------------------------

np.random.seed(42)

class_0 = np.random.randn(100, 2) + [0, 0]
class_1 = np.random.randn(100, 2) + [4, 0]
class_2 = np.random.randn(100, 2) + [2, 4]

X = np.vstack((class_0, class_1, class_2))
y = np.array([0] * 100 + [1] * 100 + [2] * 100)

print("Dataset shape:", X.shape)
print("Labels shape:", y.shape)

# ------------------------------------------------
# STEP 2: SPLIT INTO TRAINING AND TEST DATA
# ------------------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(y_train))
print("Test samples:", len(y_test))

# ------------------------------------------------
# STEP 3: CREATE THE MODEL
# ------------------------------------------------

model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=1000)
)

# ------------------------------------------------
# STEP 4: TRAIN THE MODEL
# ------------------------------------------------

model.fit(X_train, y_train)

print("\nModel training completed.")

# ------------------------------------------------
# STEP 5: MAKE PREDICTIONS
# ------------------------------------------------

y_pred = model.predict(X_test)

# Probabilities for the first five test samples
probabilities = model.predict_proba(X_test[:5])

print("\nFirst five true labels:", y_test[:5])
print("First five predictions:", y_pred[:5])

print("\nClass order:", model.classes_)
```

```
print("Probabilities for first five samples:")
print(np.round(probabilities, 3))

# ------------------------------------------------
# STEP 6: EVALUATE THE MODEL
# ------------------------------------------------

accuracy = accuracy_score(y_test, y_pred)

cm = confusion_matrix(
    y_test,
    y_pred,
    labels=[0, 1, 2]
)

print("\nTest accuracy:", round(accuracy, 4))

print("\nConfusion matrix:")
print(cm)

print("\nClassification report:")
print(
    classification_report(
        y_test,
        y_pred,
        labels=[0, 1, 2],
        zero_division=0
    )
)

# ------------------------------------------------
# STEP 7: PLOT THE CONFUSION MATRIX
# ------------------------------------------------

fig, ax = plt.subplots(figsize=(6, 5))

image = ax.imshow(cm, interpolation="nearest")
fig.colorbar(image, ax=ax)

ax.set(
    xticks=np.arange(3),
    yticks=np.arange(3),
    xticklabels=[0, 1, 2],
    yticklabels=[0, 1, 2],
    xlabel="Predicted class",
    ylabel="True class",
    title="Multiclass Confusion Matrix"
)

for row in range(3):
    for col in range(3):
        ax.text(
            col, row, str(cm[row, col]),
            ha="center", va="center"
        )

plt.tight_layout()
plt.savefig("13_confusion_matrix.png", dpi=150)
plt.show()

print("Saved 13_confusion_matrix.png")
```

```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 13_sklearn_multiclass.py
Dataset shape: (300, 2)
Labels shape: (300,)

Training samples: 240
Test samples: 60

Model training completed.

First five true labels: [0 0 2 2 2]
First five predictions: [0 0 2 2 2]

Class order: [0 1 2]
Probabilities for first five samples:
[[0.991 0.006 0.002]
 [0.998 0.001 0.   ]
 [0.007 0.001 0.992]
 [0.    0.    1.   ]
 [0.081 0.014 0.905]]

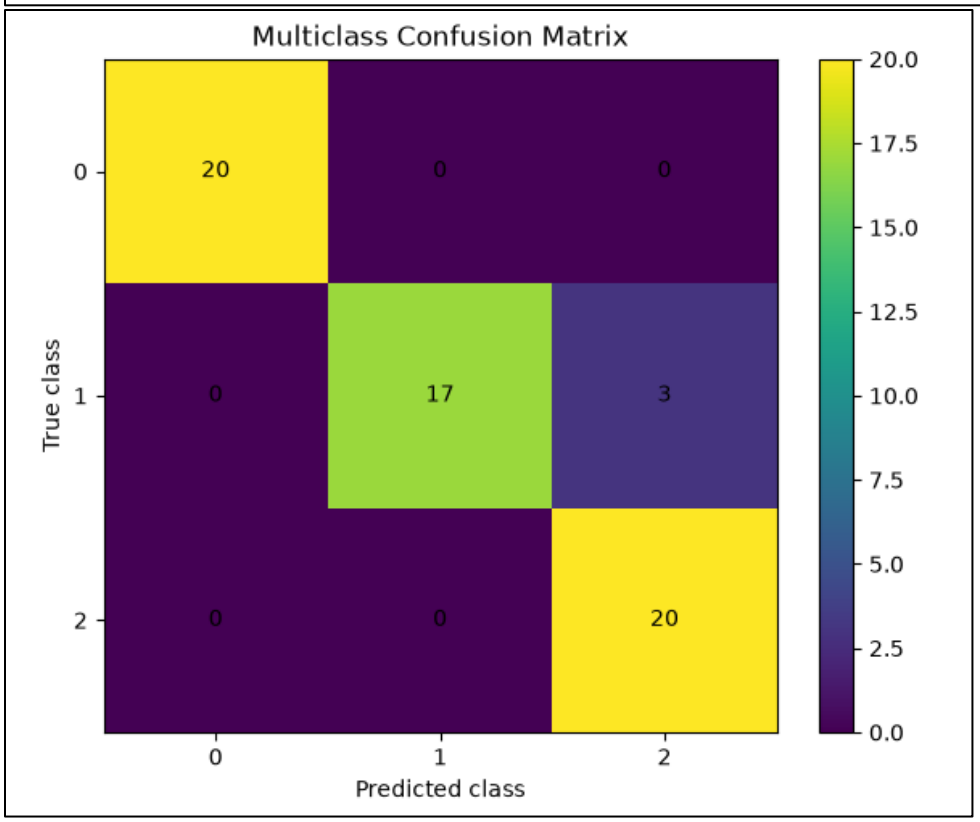
Test accuracy: 0.95

Confusion matrix:
[[20  0  0]
 [ 0 17  3]
 [ 0  0 20]]

Classification report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        20
           1       1.00      0.85      0.92        20
           2       0.87      1.00      0.93        20

    accuracy                           0.95        60
   macro avg       0.96      0.95      0.95        60
weighted avg       0.96      0.95      0.95        60
```



Nonlinear Multiclass Classification-

14_nonlinear_multiclass.py

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report


# ------------------------------------------------
# STEP 1: Generate a three-class spiral dataset
# ------------------------------------------------

def make_three_spirals(n_samples=900, noise=0.06, seed=42):
    rng = np.random.default_rng(seed)

    samples_per_class = n_samples // 3
    X_parts = []
    y_parts = []

    for class_id in range(3):
        t = np.linspace(0, 1, samples_per_class)

        # Radius gradually increases along each spiral.
        radius = 0.2 + 1.8 * t

        # Each class starts at a different angle.
        angle = 2 * np.pi * 1.25 * t + 2 * np.pi * class_id / 3

        x1 = radius * np.cos(angle)
        x2 = radius * np.sin(angle)

        points = np.column_stack((x1, x2))

        # Add random noise to make classification harder.
        points += rng.normal(
            loc=0.0,
            scale=noise,
            size=points.shape
        )

        X_parts.append(points)
        y_parts.append(
            np.full(samples_per_class, class_id)
        )

    X = np.vstack(X_parts)
    y = np.concatenate(y_parts)

    # Shuffle all samples together.
    order = rng.permutation(len(y))
    return X[order], y[order]


X, y = make_three_spirals()
```

```
print("Dataset shape:", X.shape)
print("Labels shape:", y.shape)
print("Classes:", np.unique(y))


# ------------------------------------------------
# STEP 2: Visualize the original dataset
# ------------------------------------------------

plt.figure(figsize=(7, 6))

for class_id in range(3):
    plt.scatter(
        X[y == class_id, 0],
        X[y == class_id, 1],
        label=f"Class {class_id}",
        s=18
    )

plt.title("Three-Class Spiral Dataset")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("14_spiral_dataset.png", dpi=200)
plt.show()


# ------------------------------------------------
# STEP 3: Split into training and testing data
# ------------------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))


# ------------------------------------------------
# STEP 4: Build two different classifiers
# ------------------------------------------------

linear_model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=2000)
)

nonlinear_model = make_pipeline(
    StandardScaler(),
    SVC(kernel="rbf", C=10, gamma="scale")
)


# ------------------------------------------------
# STEP 5: Train both models
# ------------------------------------------------
```

```
linear_model.fit(X_train, y_train)
nonlinear_model.fit(X_train, y_train)


# ------------------------------------------------
# STEP 6: Evaluate both models
# ------------------------------------------------

linear_predictions = linear_model.predict(X_test)
nonlinear_predictions = nonlinear_model.predict(X_test)

linear_accuracy = accuracy_score(
    y_test, linear_predictions
)

nonlinear_accuracy = accuracy_score(
    y_test, nonlinear_predictions
)

print("\n--- Test Accuracy ---")
print(f"Logistic Regression: {linear_accuracy:.4f}")
print(f"RBF SVM:             {nonlinear_accuracy:.4f}")

print("\n--- RBF SVM Classification Report ---")
print(
    classification_report(
        y_test,
        nonlinear_predictions,
        zero_division=0
    )
)


# ------------------------------------------------
# STEP 7: Create a grid for decision-boundary plots
# ------------------------------------------------

x_min, x_max = X[:, 0].min() - 0.2, X[:, 0].max() + 0.2
y_min, y_max = X[:, 1].min() - 0.2, X[:, 1].max() + 0.2

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 300),
    np.linspace(y_min, y_max, 300)
)

grid_points = np.column_stack((
    xx.ravel(),
    yy.ravel()
))


# ------------------------------------------------
# STEP 8: Plot each model's decision boundaries
# ------------------------------------------------

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

models = [
    (linear_model, "Logistic Regression (Linear)"),
    (nonlinear_model, "SVM with RBF Kernel (Nonlinear)")
]
```

```
for ax, (model, title) in zip(axes, models):

    # Predict the class at every grid location.
    grid_predictions = model.predict(grid_points)

    # Reshape predictions into the grid's 2D shape.
    regions = grid_predictions.reshape(xx.shape)

    ax.contourf(
        xx,
        yy,
        regions,
        alpha=0.25,
        levels=[-0.5, 0.5, 1.5, 2.5]
    )

    # Overlay the actual data points.
    for class_id in range(3):
        ax.scatter(
            X_test[y_test == class_id, 0],
            X_test[y_test == class_id, 1],
            label=f"Class {class_id}",
            s=18
        )

    ax.set_title(title)
    ax.set_xlabel("Feature 1")
    ax.set_ylabel("Feature 2")
    ax.legend()
    ax.grid(True, alpha=0.2)

plt.tight_layout()
plt.savefig("14_decision_boundaries.png", dpi=200)
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 14_nonlinear_multiclass.py
Dataset shape: (900, 2)
Labels shape: (900,)
Classes: [0 1 2]

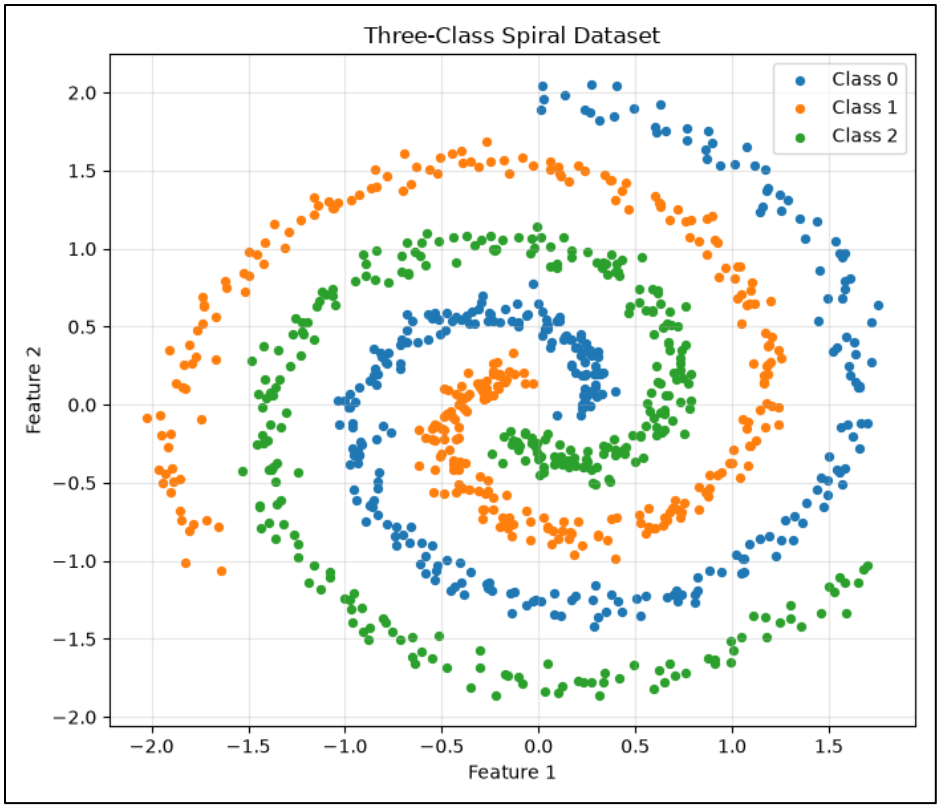
Training samples: 720
Testing samples: 180

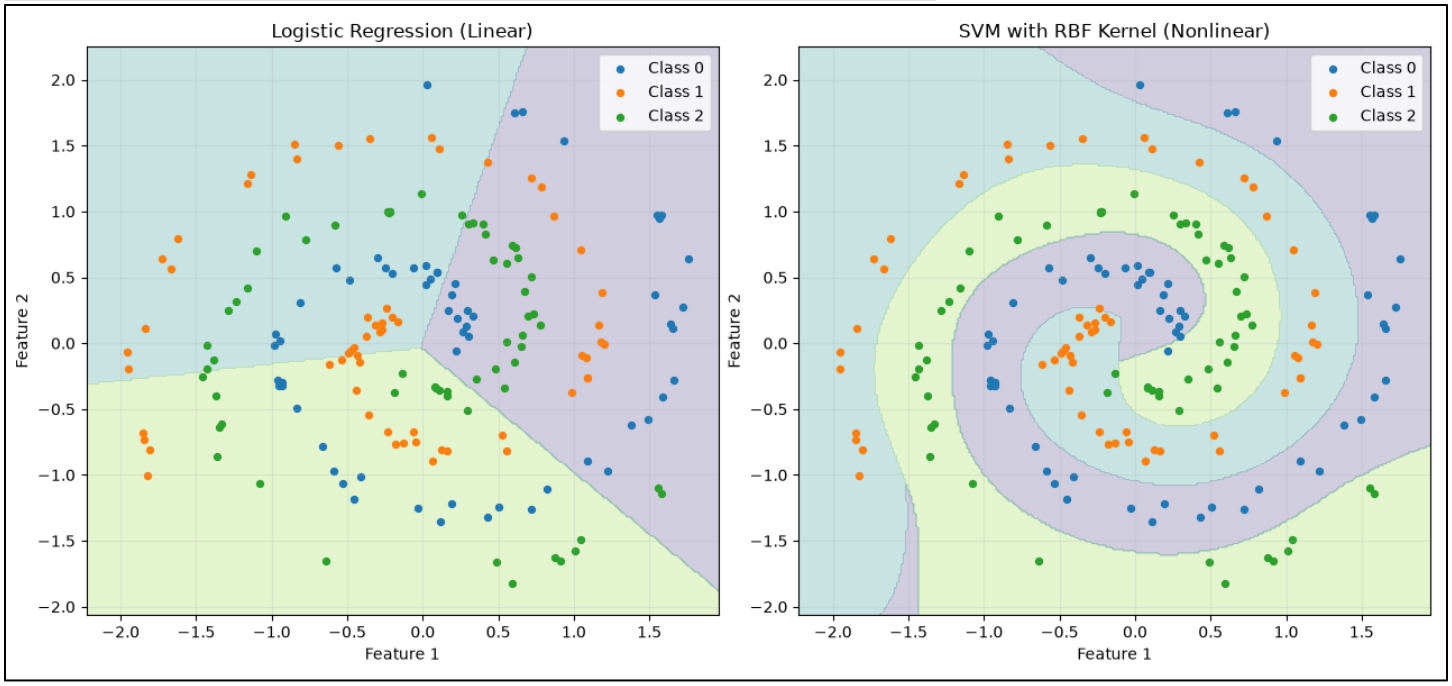
--- Test Accuracy ---
Logistic Regression: 0.4111
RBF SVM:             0.9778

--- RBF SVM Classification Report ---
              precision    recall  f1-score   support

           0       0.98      0.98      0.98        60
           1       0.97      0.98      0.98        60
           2       0.98      0.97      0.97        60

    accuracy                           0.98       180
   macro avg       0.98      0.98      0.98       180
weighted avg       0.98      0.98      0.98       180
```





Multilayer Perceptron (MLP) for Multiclass Classification-

15_mlp_from_scratch.py

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report


# ================================================
```

```
# STEP 1: CREATE A THREE-CLASS SPIRAL DATASET
# ================================================

def make_three_spirals(samples_per_class=300, noise=0.06, seed=42):
    rng = np.random.default_rng(seed)

    X_parts = []
    y_parts = []

    for class_id in range(3):
        t = np.linspace(0, 1, samples_per_class)

        radius = 0.2 + 1.8 * t
        angle = 2 * np.pi * 1.25 * t + 2 * np.pi * class_id / 3

        x1 = radius * np.cos(angle)
        x2 = radius * np.sin(angle)

        points = np.column_stack((x1, x2))
        points += rng.normal(0, noise, points.shape)

        X_parts.append(points)
        y_parts.append(
            np.full(samples_per_class, class_id)
        )

    X = np.vstack(X_parts)
    y = np.concatenate(y_parts)

    order = rng.permutation(len(y))
    return X[order], y[order]


X, y = make_three_spirals()

print("X shape:", X.shape)
print("y shape:", y.shape)


# ================================================
# STEP 2: SPLIT THE DATA
# ================================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)


# ================================================
# STEP 3: STANDARDIZE THE FEATURES
# ================================================

# Calculate statistics from training data only.
mean = X_train.mean(axis=0)
std = X_train.std(axis=0)

X_train = (X_train - mean) / std
X_test = (X_test - mean) / std
```

```
# ================================================
# STEP 4: ONE-HOT ENCODE THE CLASS LABELS
# ================================================

num_classes = 3

Y_train = np.eye(num_classes)[y_train]

print("One-hot label shape:", Y_train.shape)


# ================================================
# STEP 5: INITIALIZE THE NETWORK PARAMETERS
# ================================================

rng = np.random.default_rng(1)

input_size = 2
hidden_size = 16
output_size = 3

# Input -> hidden layer
W1 = rng.normal(
    0, np.sqrt(2 / input_size),
    (input_size, hidden_size)
)
b1 = np.zeros((1, hidden_size))

# Hidden layer -> output layer
W2 = rng.normal(
    0, np.sqrt(2 / hidden_size),
    (hidden_size, output_size)
)
b2 = np.zeros((1, output_size))


# ================================================
# STEP 6: DEFINE ACTIVATION AND LOSS FUNCTIONS
# ================================================

def relu(z):
    return np.maximum(0, z)


def softmax(z):
    # Subtract row-wise maximum for numerical stability.
    z = z - np.max(z, axis=1, keepdims=True)

    exp_z = np.exp(z)

    return exp_z / np.sum(
        exp_z, axis=1, keepdims=True
    )


def cross_entropy(probabilities, labels):
    epsilon = 1e-12

    correct_class_probs = np.sum(
        labels * probabilities, axis=1
    )
```

```
    return -np.mean(
        np.log(correct_class_probs + epsilon)
    )


# ================================================
# STEP 7: FORWARD PROPAGATION
# ================================================

def forward(X):
    # First layer: weighted sum
    Z1 = X @ W1 + b1

    # First layer: nonlinear activation
    A1 = relu(Z1)

    # Output layer: class scores
    Z2 = A1 @ W2 + b2

    # Convert scores into probabilities
    P = softmax(Z2)

    return Z1, A1, Z2, P


# ================================================
# STEP 8: TRAIN THE NETWORK
# ================================================

learning_rate = 0.05
epochs = 2500
m = len(X_train)

loss_history = []

for epoch in range(epochs):

    # ----- Forward pass -----
    Z1, A1, Z2, P = forward(X_train)

    loss = cross_entropy(P, Y_train)
    loss_history.append(loss)

    # ----- Backward pass -----

    # Gradient of cross-entropy + softmax
    dZ2 = (P - Y_train) / m

    # Gradients for output-layer parameters
    dW2 = A1.T @ dZ2
    db2 = np.sum(dZ2, axis=0, keepdims=True)

    # Send gradient back into the hidden layer
    dA1 = dZ2 @ W2.T

    # Gradient through ReLU
    dZ1 = dA1 * (Z1 > 0)

    # Gradients for hidden-layer parameters
    dW1 = X_train.T @ dZ1
    db1 = np.sum(dZ1, axis=0, keepdims=True)

    # ----- Gradient descent updates -----
```

```
    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1

    W2 -= learning_rate * dW2
    b2 -= learning_rate * db2

    if (epoch + 1) % 250 == 0:
        print(
            f"Epoch {epoch + 1:4d} | "
            f"Loss: {loss:.4f}"
        )


# ================================================
# STEP 9: EVALUATE ON TEST DATA
# ================================================

_, _, _, test_probabilities = forward(X_test)

y_pred = np.argmax(test_probabilities, axis=1)

test_accuracy = accuracy_score(y_test, y_pred)

print("\nTest accuracy:", round(test_accuracy, 4))

print("\nClassification report:")
print(
    classification_report(
        y_test,
        y_pred,
        zero_division=0
    )
)


# ================================================
# STEP 10: PLOT THE TRAINING LOSS
# ================================================

plt.figure(figsize=(7, 5))

plt.plot(loss_history)

plt.title("MLP Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Cross-Entropy Loss")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("15_training_loss.png", dpi=200)
plt.show()


# ================================================
# STEP 11: VISUALIZE THE DECISION BOUNDARIES
# ================================================

x_min, x_max = X[:, 0].min() - 0.3, X[:, 0].max() + 0.3
y_min, y_max = X[:, 1].min() - 0.3, X[:, 1].max() + 0.3

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 300),
    np.linspace(y_min, y_max, 300)
)
```

```
grid = np.column_stack((xx.ravel(), yy.ravel()))

# Use training statistics to scale the grid.
grid_scaled = (grid - mean) / std

_, _, _, grid_probs = forward(grid_scaled)

grid_classes = np.argmax(grid_probs, axis=1)
regions = grid_classes.reshape(xx.shape)

plt.figure(figsize=(8, 6))

plt.contourf(
    xx,
    yy,
    regions,
    levels=[-0.5, 0.5, 1.5, 2.5],
    alpha=0.3
)

for class_id in range(3):
    plt.scatter(
        X_test[y_test == class_id, 0],
        X_test[y_test == class_id, 1],
        label=f"Class {class_id}",
        s=20
    )

plt.title("MLP Decision Boundaries")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.grid(True, alpha=0.2)
plt.tight_layout()
plt.savefig("15_mlp_decision_boundaries.png", dpi=200)
plt.show()
```

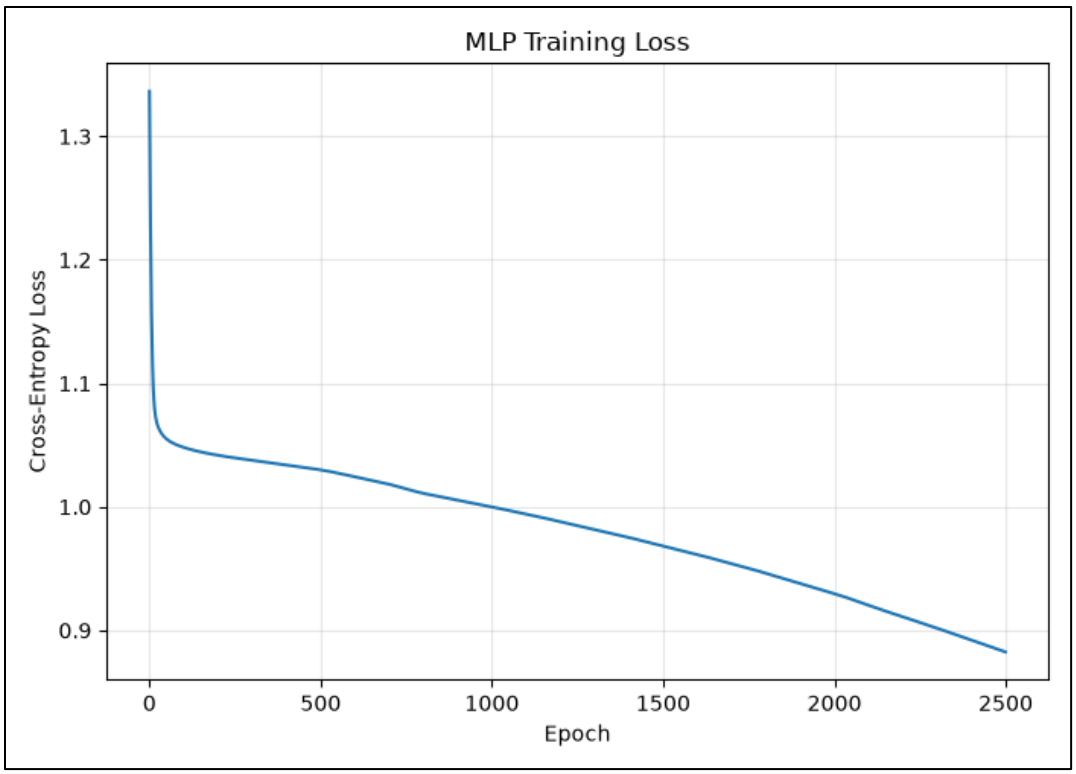
```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 15_mlp_from_scratch.py
X shape: (900, 2)
y shape: (900,)
One-hot label shape: (720, 3)
Epoch  250 | Loss: 1.0396
Epoch  500 | Loss: 1.0299
Epoch  750 | Loss: 1.0145
Epoch 1000 | Loss: 1.0000
Epoch 1250 | Loss: 0.9848
Epoch 1500 | Loss: 0.9683
Epoch 1750 | Loss: 0.9504
Epoch 2000 | Loss: 0.9297
Epoch 2250 | Loss: 0.9064
Epoch 2500 | Loss: 0.8825

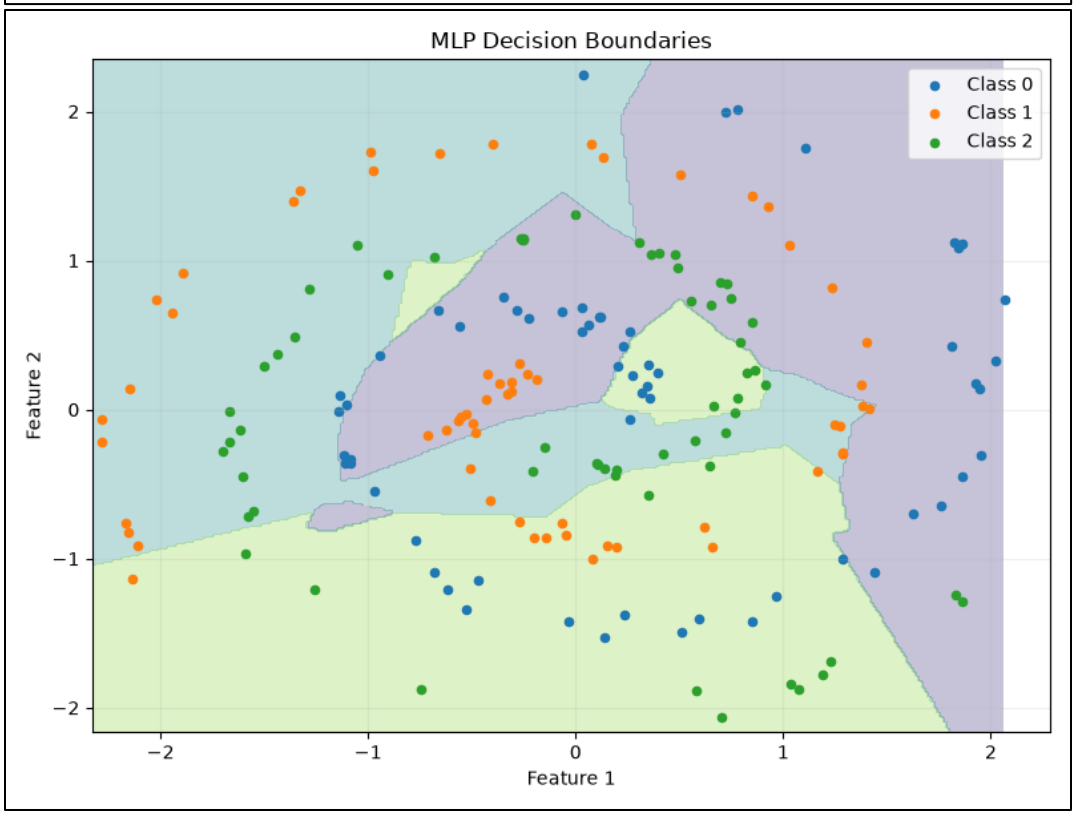
Test accuracy: 0.4778

Classification report:
              precision    recall  f1-score   support

           0       0.50      0.62      0.55        60
           1       0.52      0.47      0.49        60
           2       0.40      0.35      0.38        60

    accuracy                           0.48       180
   macro avg       0.47      0.48      0.47       180
weighted avg       0.47      0.48      0.47       180
```





16_improve_mlp.py

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score


# ================================================
```

```
# STEP 1: GENERATE THREE-CLASS SPIRAL DATA
# ================================================

def make_three_spirals(samples_per_class=300, noise=0.06, seed=42):
    rng = np.random.default_rng(seed)

    X_parts = []
    y_parts = []

    for class_id in range(3):
        t = np.linspace(0, 1, samples_per_class)

        radius = 0.2 + 1.8 * t
        angle = 2 * np.pi * 1.25 * t + 2 * np.pi * class_id / 3

        points = np.column_stack((
            radius * np.cos(angle),
            radius * np.sin(angle)
        ))

        points += rng.normal(0, noise, points.shape)

        X_parts.append(points)
        y_parts.append(
            np.full(samples_per_class, class_id)
        )

    X = np.vstack(X_parts)
    y = np.concatenate(y_parts)

    order = rng.permutation(len(y))
    return X[order], y[order]


X, y = make_three_spirals()


# ================================================
# STEP 2: CREATE TRAIN, VALIDATION, AND TEST SETS
# ================================================

X_train_all, X_test, y_train_all, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

X_train, X_val, y_train, y_val = train_test_split(
    X_train_all,
    y_train_all,
    test_size=0.20,
    random_state=21,
    stratify=y_train_all
)

# 64% training, 16% validation, 20% testing.
# Fit scaling statistics on training data only.
mean = X_train.mean(axis=0)
std = X_train.std(axis=0)

X_train = (X_train - mean) / std
X_val = (X_val - mean) / std
X_test = (X_test - mean) / std
```

```
num_classes = 3

Y_train = np.eye(num_classes)[y_train]
Y_val = np.eye(num_classes)[y_val]


# ================================================
# STEP 3: ACTIVATION, SOFTMAX, AND LOSS
# ================================================

def relu(z):
    return np.maximum(0, z)


def softmax(z):
    z = z - np.max(z, axis=1, keepdims=True)
    exp_z = np.exp(z)
    return exp_z / np.sum(exp_z, axis=1, keepdims=True)


def cross_entropy(probabilities, one_hot_labels):
    eps = 1e-12

    correct_probabilities = np.sum(
        probabilities * one_hot_labels,
        axis=1
    )

    return -np.mean(np.log(correct_probabilities + eps))


# ================================================
# STEP 4: TRAIN ONE MLP CONFIGURATION
# ================================================

def train_mlp(
    hidden_size,
    learning_rate,
    epochs=1000,
    seed=1
):
    rng = np.random.default_rng(seed)

    input_size = 2
    output_size = 3

    # He-style initialization for ReLU.
    W1 = rng.normal(
        0,
        np.sqrt(2 / input_size),
        (input_size, hidden_size)
    )
    b1 = np.zeros((1, hidden_size))

    W2 = rng.normal(
        0,
        np.sqrt(2 / hidden_size),
        (hidden_size, output_size)
    )
    b2 = np.zeros((1, output_size))

    n = len(X_train)

    history = {
        "train_loss": [],
        "val_loss": [],
```

```
        "train_accuracy": [],
        "val_accuracy": []
    }

    best_val_loss = float("inf")
    best_epoch = 0
    best_parameters = None

    for epoch in range(epochs):

        # ----- Forward pass on training data -----
        Z1 = X_train @ W1 + b1
        A1 = relu(Z1)

        Z2 = A1 @ W2 + b2
        P = softmax(Z2)

        loss = cross_entropy(P, Y_train)

        # ----- Backpropagation -----
        dZ2 = (P - Y_train) / n

        dW2 = A1.T @ dZ2
        db2 = np.sum(dZ2, axis=0, keepdims=True)

        dA1 = dZ2 @ W2.T
        dZ1 = dA1 * (Z1 > 0)

        dW1 = X_train.T @ dZ1
        db1 = np.sum(dZ1, axis=0, keepdims=True)

        # ----- Gradient descent -----
        W1 -= learning_rate * dW1
        b1 -= learning_rate * db1

        W2 -= learning_rate * dW2
        b2 -= learning_rate * db2

        # ----- Evaluate after the parameter update -----
        train_prob = softmax(
            relu(X_train @ W1 + b1) @ W2 + b2
        )

        val_prob = softmax(
            relu(X_val @ W1 + b1) @ W2 + b2
        )

        train_loss = cross_entropy(train_prob, Y_train)
        val_loss = cross_entropy(val_prob, Y_val)

        train_pred = np.argmax(train_prob, axis=1)
        val_pred = np.argmax(val_prob, axis=1)

        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_loss)

        history["train_accuracy"].append(
            accuracy_score(y_train, train_pred)
        )
        history["val_accuracy"].append(
            accuracy_score(y_val, val_pred)
        )

        # Keep the parameters with the lowest validation loss.
        if val_loss < best_val_loss:
            best_val_loss = val_loss
```

```
            best_epoch = epoch + 1

            best_parameters = (
                W1.copy(),
                b1.copy(),
                W2.copy(),
                b2.copy()
            )

    return {
        "hidden_size": hidden_size,
        "learning_rate": learning_rate,
        "history": history,
        "best_val_loss": best_val_loss,
        "best_epoch": best_epoch,
        "parameters": best_parameters
    }


# ================================================
# STEP 5: EXPERIMENT WITH HIDDEN SIZE AND LEARNING RATE
# ================================================

hidden_sizes = [8, 16, 32]
learning_rates = [0.01, 0.05, 0.20]

results = []

for hidden_size in hidden_sizes:
    for lr in learning_rates:

        result = train_mlp(
            hidden_size=hidden_size,
            learning_rate=lr,
            epochs=1000,
            seed=1
        )

        results.append(result)

        h = result["history"]

        print(
            f"Hidden={hidden_size:2d}, "
            f"LR={lr:.2f}, "
            f"Best epoch={result['best_epoch']:4d}, "
            f"Val accuracy="
            f"{max(h['val_accuracy']):.4f}, "
            f"Best val loss={result['best_val_loss']:.4f}"
        )


# ================================================
# STEP 6: SELECT THE BEST VALIDATION CONFIGURATION
# ================================================

best_result = min(
    results,
    key=lambda result: result["best_val_loss"]
)

print("\nSelected configuration:")
print("Hidden neurons:", best_result["hidden_size"])
print("Learning rate:", best_result["learning_rate"])
print("Best epoch:", best_result["best_epoch"])
print("Best validation loss:", round(best_result["best_val_loss"], 4))
```

```
# ================================================
# STEP 7: EVALUATE ON THE HELD-OUT TEST SET
# ================================================

W1, b1, W2, b2 = best_result["parameters"]

test_probabilities = softmax(
    relu(X_test @ W1 + b1) @ W2 + b2
)

test_predictions = np.argmax(test_probabilities, axis=1)

test_accuracy = accuracy_score(y_test, test_predictions)

print("\nFinal test accuracy:", round(test_accuracy, 4))


# ================================================
# STEP 8: PLOT TRAINING AND VALIDATION LOSS
# ================================================

history = best_result["history"]

plt.figure(figsize=(8, 5))
plt.plot(history["train_loss"], label="Training loss")
plt.plot(history["val_loss"], label="Validation loss")

plt.axvline(
    best_result["best_epoch"] - 1,
    linestyle="--",
    label="Selected epoch"
)

plt.xlabel("Epoch")
plt.ylabel("Cross-entropy loss")
plt.title("Training vs Validation Loss")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("16_loss_comparison.png", dpi=200)
plt.show()


# ================================================
# STEP 9: COMPARE VALIDATION ACCURACY
# ================================================

plt.figure(figsize=(8, 5))

labels = [
    f"H={r['hidden_size']}, LR={r['learning_rate']}"
    for r in results
]

accuracies = [
    max(r["history"]["val_accuracy"])
    for r in results
]

plt.bar(labels, accuracies)
plt.ylabel("Best validation accuracy")
plt.xlabel("Network configuration")
plt.title("MLP Hyperparameter Comparison")
plt.xticks(rotation=45, ha="right")
```

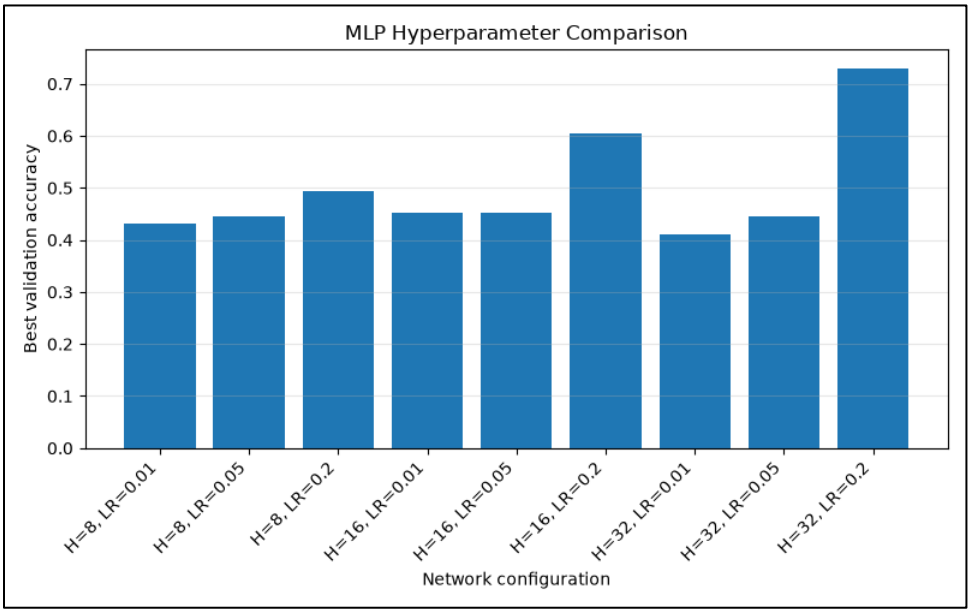
```
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.savefig("16_hyperparameter_comparison.png", dpi=200)
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 16_improve_mlp.py
Hidden= 8, LR=0.01, Best epoch=1000, Val accuracy=0.4306, Best val loss=1.0640
Hidden= 8, LR=0.05, Best epoch=1000, Val accuracy=0.4444, Best val loss=1.0475
Hidden= 8, LR=0.20, Best epoch=1000, Val accuracy=0.4931, Best val loss=0.9582
Hidden=16, LR=0.01, Best epoch= 233, Val accuracy=0.4514, Best val loss=1.0553
Hidden=16, LR=0.05, Best epoch=1000, Val accuracy=0.4514, Best val loss=1.0339
Hidden=16, LR=0.20, Best epoch=1000, Val accuracy=0.6042, Best val loss=0.7749
Hidden=32, LR=0.01, Best epoch=1000, Val accuracy=0.4097, Best val loss=1.0566
Hidden=32, LR=0.05, Best epoch=1000, Val accuracy=0.4444, Best val loss=1.0257
Hidden=32, LR=0.20, Best epoch=1000, Val accuracy=0.7292, Best val loss=0.7079

Selected configuration:
Hidden neurons: 32
Learning rate: 0.2
Best epoch: 1000
Best validation loss: 0.7079

Final test accuracy: 0.7611
```





Reducing overfitting-

A. **L2 regularization:** Add a penalty proportional to the squared magnitude of the weights.

$$L_{\text{total}} = L_{\text{data}} + \frac{\lambda}{2m}\sum W^2$$

This encourages smaller weights and can reduce overly complex fits.

B. Dropout: Randomly set some hidden activations to zero during training. The network must learn useful representations without depending too heavily on any one neuron.

C. Early stopping: Stop training when validation performance no longer improves, and retain the best checkpoint.

17_regularization_dropout.py

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score


# ================================================
# STEP 1: GENERATE THE THREE-CLASS SPIRAL DATASET
# ================================================

def make_three_spirals(samples_per_class=300, noise=0.12, seed=42):
    rng = np.random.default_rng(seed)

    X_parts = []
    y_parts = []

    for class_id in range(3):
        t = np.linspace(0, 1, samples_per_class)

        radius = 0.2 + 1.8 * t
        angle = 2 * np.pi * 1.25 * t + 2 * np.pi * class_id / 3

        points = np.column_stack((
            radius * np.cos(angle),
            radius * np.sin(angle)
        ))

        points += rng.normal(0, noise, points.shape)

        X_parts.append(points)
        y_parts.append(
            np.full(samples_per_class, class_id)
        )

    X = np.vstack(X_parts)
    y = np.concatenate(y_parts)

    order = rng.permutation(len(y))
    return X[order], y[order]


X, y = make_three_spirals()


# ================================================
# STEP 2: TRAIN, VALIDATION, AND TEST SPLITS
# ================================================
```

```
X_train_all, X_test, y_train_all, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

X_train, X_val, y_train, y_val = train_test_split(
    X_train_all, y_train_all,
    test_size=0.20,
    random_state=21,
    stratify=y_train_all
)

# Standardize using training data only.
mean = X_train.mean(axis=0)
std = X_train.std(axis=0)

X_train = (X_train - mean) / std
X_val = (X_val - mean) / std
X_test = (X_test - mean) / std

num_classes = 3
Y_train = np.eye(num_classes)[y_train]
Y_val = np.eye(num_classes)[y_val]


# ================================================
# STEP 3: BASIC FUNCTIONS
# ================================================

def relu(z):
    return np.maximum(0, z)


def softmax(z):
    z = z - np.max(z, axis=1, keepdims=True)
    exp_z = np.exp(z)
    return exp_z / np.sum(exp_z, axis=1, keepdims=True)


def cross_entropy(probabilities, one_hot_labels):
    eps = 1e-12

    correct_probabilities = np.sum(
        probabilities * one_hot_labels,
        axis=1
    )

    return -np.mean(np.log(correct_probabilities + eps))


# ================================================
# STEP 4: FORWARD PASS
# ================================================

def forward(X, params, training=False, dropout_rate=0.0, rng=None):
    W1, b1, W2, b2 = params

    Z1 = X @ W1 + b1
    A1 = relu(Z1)

    dropout_mask = None
```

```
    if training and dropout_rate > 0:
        # Inverted dropout:
        # retained activations are scaled by 1/(1-rate).
        keep_probability = 1.0 - dropout_rate

        dropout_mask = (
            rng.random(A1.shape) < keep_probability
        ).astype(float)

        A1 = A1 * dropout_mask / keep_probability

    Z2 = A1 @ W2 + b2
    P = softmax(Z2)

    cache = (Z1, A1, dropout_mask)

    return P, cache


# ================================================
# STEP 5: INITIALIZE PARAMETERS
# ================================================

def initialize_network(hidden_size=32, seed=1):
    rng = np.random.default_rng(seed)

    W1 = rng.normal(
        0, np.sqrt(2 / 2),
        (2, hidden_size)
    )
    b1 = np.zeros((1, hidden_size))

    W2 = rng.normal(
        0, np.sqrt(2 / hidden_size),
        (hidden_size, 3)
    )
    b2 = np.zeros((1, 3))

    return [W1, b1, W2, b2]


# ================================================
# STEP 6: TRAIN ONE MODEL
# ================================================

def train_model(
    name,
    l2_lambda=0.0,
    dropout_rate=0.0,
    epochs=1500,
    learning_rate=0.03,
    seed=1
):
    params = initialize_network(hidden_size=32, seed=seed)
    rng = np.random.default_rng(seed + 100)

    W1, b1, W2, b2 = params
    m = len(X_train)

    history = {
        "train_loss": [],
        "val_loss": [],
```

```
        "train_accuracy": [],
        "val_accuracy": []
    }

    best_val_loss = float("inf")
    best_epoch = 0
    best_params = None

    for epoch in range(epochs):

        # ----- Forward pass with dropout during training -----
        P, cache = forward(
            X_train,
            params,
            training=True,
            dropout_rate=dropout_rate,
            rng=rng
        )

        Z1, A1, dropout_mask = cache

        # Data loss plus L2 penalty on weights.
        data_loss = cross_entropy(P, Y_train)

        regularization_loss = (
            l2_lambda / (2 * m)
        ) * (
            np.sum(W1 ** 2) + np.sum(W2 ** 2)
        )

        # ----- Backpropagation -----
        dZ2 = (P - Y_train) / m

        dW2 = A1.T @ dZ2 + (l2_lambda / m) * W2
        db2 = np.sum(dZ2, axis=0, keepdims=True)

        dA1 = dZ2 @ W2.T

        # Backpropagate through dropout using the same mask.
        if dropout_mask is not None:
            keep_probability = 1.0 - dropout_rate
            dA1 = (
                dA1 * dropout_mask / keep_probability
            )

        # Backpropagate through ReLU.
        dZ1 = dA1 * (Z1 > 0)

        dW1 = X_train.T @ dZ1 + (l2_lambda / m) * W1
        db1 = np.sum(dZ1, axis=0, keepdims=True)

        # ----- Gradient descent -----
        W1 -= learning_rate * dW1
        b1 -= learning_rate * db1

        W2 -= learning_rate * dW2
        b2 -= learning_rate * db2

        # ----- Evaluate without dropout -----
        # Validation uses the complete network.
        train_prob, _ = forward(X_train, params)
        val_prob, _ = forward(X_val, params)
```

```
        train_data_loss = cross_entropy(train_prob, Y_train)
        val_data_loss = cross_entropy(val_prob, Y_val)

        train_total_loss = train_data_loss + (
            l2_lambda / (2 * m)
        ) * (
            np.sum(W1 ** 2) + np.sum(W2 ** 2)
        )

        val_total_loss = val_data_loss + (
            l2_lambda / (2 * m)
        ) * (
            np.sum(W1 ** 2) + np.sum(W2 ** 2)
        )

        train_pred = np.argmax(train_prob, axis=1)
        val_pred = np.argmax(val_prob, axis=1)

        history["train_loss"].append(train_total_loss)
        history["val_loss"].append(val_total_loss)

        history["train_accuracy"].append(
            accuracy_score(y_train, train_pred)
        )
        history["val_accuracy"].append(
            accuracy_score(y_val, val_pred)
        )

        # Save the parameters with the lowest validation loss.
        if val_total_loss < best_val_loss:
            best_val_loss = val_total_loss
            best_epoch = epoch + 1
            best_params = [p.copy() for p in params]

    print(f"\n{name}")
    print("Best epoch:", best_epoch)
    print("Best validation loss:", round(best_val_loss, 4))
    print(
        "Best validation accuracy:",
        round(max(history["val_accuracy"]), 4)
    )

    return {
        "name": name,
        "history": history,
        "params": best_params,
        "best_epoch": best_epoch,
        "best_val_loss": best_val_loss
    }


# ================================================
# STEP 7: TRAIN THREE CONFIGURATIONS
# ================================================

baseline = train_model(
    "Baseline MLP",
    l2_lambda=0.0,
    dropout_rate=0.0
)

l2_model = train_model(
    "L2 Regularization",
```

```
    l2_lambda=0.01,
    dropout_rate=0.0
)

dropout_model = train_model(
    "Dropout",
    l2_lambda=0.0,
    dropout_rate=0.25
)

models = [baseline, l2_model, dropout_model]


# ================================================
# STEP 8: FINAL TEST EVALUATION
# ================================================

print("\n--- Final Test Results ---")

for model in models:
    probabilities, _ = forward(
        X_test,
        model["params"],
        training=False
    )

    predictions = np.argmax(probabilities, axis=1)
    accuracy = accuracy_score(y_test, predictions)

    print(f"{model['name']}: {accuracy:.4f}")


# ================================================
# STEP 9: PLOT VALIDATION LOSSES
# ================================================

plt.figure(figsize=(9, 5))

for model in models:
    plt.plot(
        model["history"]["val_loss"],
        label=model["name"]
    )

plt.xlabel("Epoch")
plt.ylabel("Validation loss")
plt.title("Regularization Comparison")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("17_validation_loss.png", dpi=200)
plt.show()


# ================================================
# STEP 10: PLOT VALIDATION ACCURACIES
# ================================================

plt.figure(figsize=(9, 5))

for model in models:
    plt.plot(
        model["history"]["val_accuracy"],
```

```
        label=model["name"]
    )

plt.xlabel("Epoch")
plt.ylabel("Validation accuracy")
plt.title("Validation Accuracy Comparison")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("17_validation_accuracy.png", dpi=200)
plt.show()
```

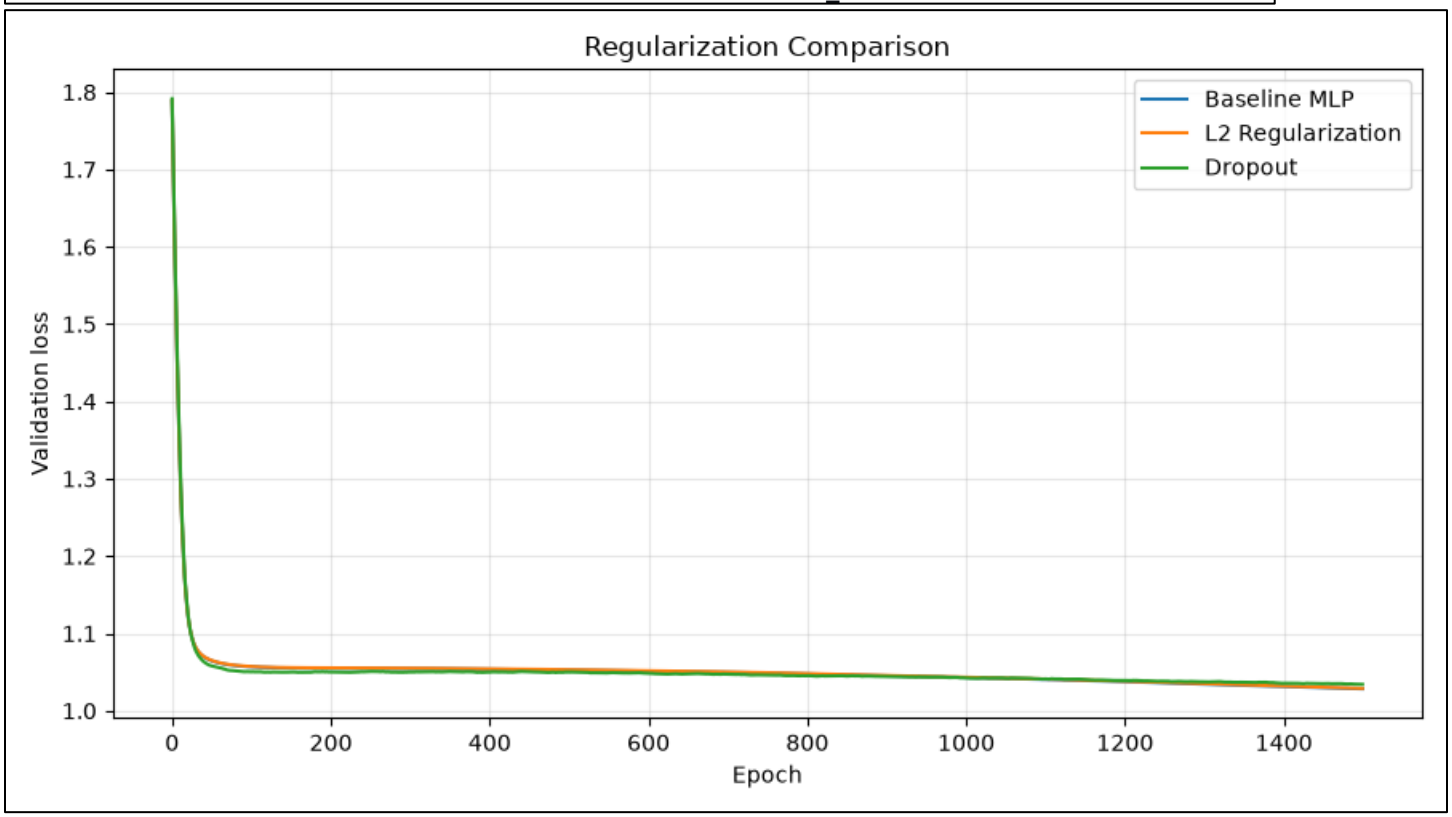
```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 17_regularization_dropout.py

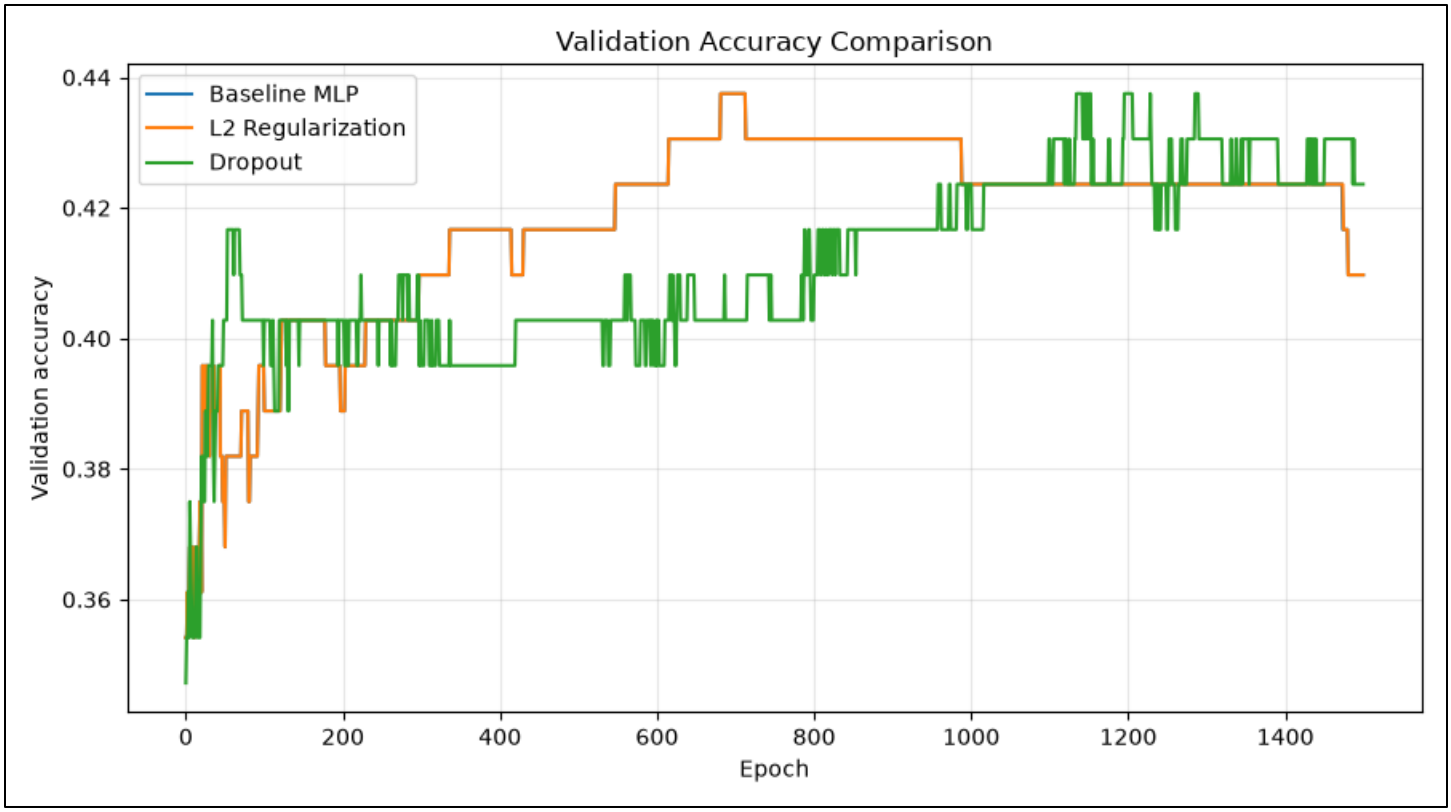
Baseline MLP
Best epoch: 1500
Best validation loss: 1.0284
Best validation accuracy: 0.4375

L2 Regularization
Best epoch: 1500
Best validation loss: 1.0289
Best validation accuracy: 0.4375

Dropout
Best epoch: 1499
Best validation loss: 1.034
Best validation accuracy: 0.4375

--- Final Test Results ---
Baseline MLP: 0.4333
L2 Regularization: 0.4333
Dropout: 0.4444
```





Batch Normalization and Learning-Rate Scheduling-

Suppose a hidden neuron produces these four activations: $x = [2,\ 4,\ 6,\ 8]$

The mean is: $\mu = \frac{2+4+6+8}{4} = 5$

The population variance is: $\sigma^2 = \frac{(2-5)^2+(4-5)^2+(6-5)^2+(8-5)^2}{4} = 5$

Batch normalization first standardizes the activations: $\hat{x}_i = \frac{x_i-\mu}{\sqrt{\sigma^2+\epsilon}}$

Here, ϵ is a small number added for numerical stability.

It then applies a learnable scale and shift:

$$y_i = \gamma\hat{x}_i + \beta$$

- γ controls the scale.
- β controls the shift.
- ϵ prevents division by zero.

The network learns γ and β along with its other parameters. Consequently, batch normalization does not simply force every layer's output to remain standardized; it gives the network the ability to learn an appropriate scale and offset.

Gradient descent updates a parameter using: $\theta_{t+1} = \theta_t - \eta_t\nabla_\theta L$

In earlier lessons, the learning rate η was constant. This time, we'll gradually reduce it.

For example, a step-decay schedule could use:

Epoch range	Learning rate
1–300	0.05
301–600	0.025
601–900	0.0125
901 onward	0.00625

The learning rate is halved at each milestone. This is just one possible schedule; other common choices include cosine decay and exponential decay.

18_batchnorm_lr_schedule.py

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score


# ====================================================
# STEP 1: CREATE THE DATASET
# ====================================================

def make_three_spirals(samples_per_class=300, noise=0.08, seed=42):
    rng = np.random.default_rng(seed)
    X_parts, y_parts = [], []

    for class_id in range(3):
        t = np.linspace(0, 1, samples_per_class)
        radius = 0.2 + 1.8 * t
        angle = 2 * np.pi * 1.25 * t + 2 * np.pi * class_id / 3

        points = np.column_stack((
            radius * np.cos(angle),
            radius * np.sin(angle)
        ))
        points += rng.normal(0, noise, points.shape)

        X_parts.append(points)
        y_parts.append(np.full(samples_per_class, class_id))

    X = np.vstack(X_parts)
    y = np.concatenate(y_parts)

    order = rng.permutation(len(y))
    return X[order], y[order]


X, y = make_three_spirals()

X_train_all, X_test, y_train_all, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

X_train, X_val, y_train, y_val = train_test_split(
    X_train_all, y_train_all,
    test_size=0.20, random_state=21, stratify=y_train_all
)

# Standardize input features using training data only.
mean_x = X_train.mean(axis=0)
std_x = X_train.std(axis=0)

X_train = (X_train - mean_x) / std_x
X_val = (X_val - mean_x) / std_x
X_test = (X_test - mean_x) / std_x

Y_train = np.eye(3)[y_train]
Y_val = np.eye(3)[y_val]


# ====================================================
# STEP 2: HELPER FUNCTIONS
```

```
# ================================================

def relu(z):
    return np.maximum(0, z)


def softmax(z):
    z = z - np.max(z, axis=1, keepdims=True)
    e = np.exp(z)
    return e / np.sum(e, axis=1, keepdims=True)


def cross_entropy(p, y_one_hot):
    eps = 1e-12
    correct_p = np.sum(p * y_one_hot, axis=1)
    return -np.mean(np.log(correct_p + eps))


# ================================================
# STEP 3: BUILD AND TRAIN THE MLP
# ================================================

def train_mlp(use_batchnorm, use_schedule, epochs=1000, seed=1):
    rng = np.random.default_rng(seed)

    input_size = 2
    hidden_size = 32
    output_size = 3
    m = len(X_train)

    W1 = rng.normal(
        0, np.sqrt(2 / input_size),
        (input_size, hidden_size)
    )
    b1 = np.zeros((1, hidden_size))

    W2 = rng.normal(
        0, np.sqrt(2 / hidden_size),
        (hidden_size, output_size)
    )
    b2 = np.zeros((1, output_size))

    # Batch normalization parameters.
    gamma = np.ones((1, hidden_size))
    beta = np.zeros((1, hidden_size))

    # Running statistics used for evaluation.
    running_mean = np.zeros((1, hidden_size))
    running_var = np.ones((1, hidden_size))
    momentum = 0.9
    eps = 1e-5

    train_losses = []
    val_losses = []
    train_accuracies = []
    val_accuracies = []
    learning_rates = []

    best_val_loss = float("inf")
    best_epoch = 0
    best_params = None

    for epoch in range(epochs):
```

```
        # ----- Learning-rate schedule -----
        if use_schedule:
            learning_rate = 0.05 * (0.5 ** (epoch // 300))
        else:
            learning_rate = 0.05

        learning_rates.append(learning_rate)

        # ----- Forward pass: hidden layer -----
        Z1 = X_train @ W1 + b1

        if use_batchnorm:
            batch_mean = Z1.mean(axis=0, keepdims=True)
            batch_var = Z1.var(axis=0, keepdims=True)

            Xhat = (Z1 - batch_mean) / np.sqrt(batch_var + eps)
            Z1_norm = gamma * Xhat + beta

            # Update running statistics for evaluation.
            running_mean = (
                momentum * running_mean
                + (1 - momentum) * batch_mean
            )
            running_var = (
                momentum * running_var
                + (1 - momentum) * batch_var
            )
        else:
            Xhat = None
            Z1_norm = Z1

        A1 = relu(Z1_norm)

        # ----- Forward pass: output layer -----
        Z2 = A1 @ W2 + b2
        P = softmax(Z2)

        # ----- Backpropagation: output layer -----
        dZ2 = (P - Y_train) / m

        dW2 = A1.T @ dZ2
        db2 = np.sum(dZ2, axis=0, keepdims=True)

        dA1 = dZ2 @ W2.T
        dZ1_norm = dA1 * (Z1_norm > 0)

        # ----- Backpropagation: batch normalization -----
        if use_batchnorm:
            dgamma = np.sum(
                dZ1_norm * Xhat, axis=0, keepdims=True
            )
            dbeta = np.sum(
                dZ1_norm, axis=0, keepdims=True
            )

            dXhat = dZ1_norm * gamma
            inv_std = 1.0 / np.sqrt(batch_var + eps)

            dZ1 = (inv_std / m) * (
                m * dXhat
                - np.sum(dXhat, axis=0, keepdims=True)
                - Xhat * np.sum(
```

```
                    dXhat * Xhat, axis=0, keepdims=True
                )
            )
        else:
            dZ1 = dZ1_norm

        dW1 = X_train.T @ dZ1
        db1 = np.sum(dZ1, axis=0, keepdims=True)

        # ----- Gradient descent -----
        W1 -= learning_rate * dW1
        b1 -= learning_rate * db1
        W2 -= learning_rate * dW2
        b2 -= learning_rate * db2

        if use_batchnorm:
            gamma -= learning_rate * dgamma
            beta -= learning_rate * dbeta

        # ----- Evaluation without gradient tracking -----
        def predict_probabilities(X_input):
            Z = X_input @ W1 + b1

            if use_batchnorm:
                Z = (
                    (Z - running_mean)
                    / np.sqrt(running_var + eps)
                )
                Z = gamma * Z + beta

            A = relu(Z)
            return softmax(A @ W2 + b2)

        train_p = predict_probabilities(X_train)
        val_p = predict_probabilities(X_val)

        train_loss = cross_entropy(train_p, Y_train)
        val_loss = cross_entropy(val_p, Y_val)

        train_pred = np.argmax(train_p, axis=1)
        val_pred = np.argmax(val_p, axis=1)

        train_losses.append(train_loss)
        val_losses.append(val_loss)

        train_accuracies.append(
            accuracy_score(y_train, train_pred)
        )
        val_accuracies.append(
            accuracy_score(y_val, val_pred)
        )

        if val_loss < best_val_loss:
            best_val_loss = val_loss
            best_epoch = epoch + 1

            best_params = {
                "W1": W1.copy(),
                "b1": b1.copy(),
                "W2": W2.copy(),
                "b2": b2.copy(),
                "gamma": gamma.copy(),
                "beta": beta.copy(),
```

```
                "running_mean": running_mean.copy(),
                "running_var": running_var.copy()
            }

    return {
        "name": (
            "BatchNorm + Schedule" if use_batchnorm and use_schedule
            else "BatchNorm" if use_batchnorm
            else "Baseline"
        ),
        "use_batchnorm": use_batchnorm,
        "history": {
            "train_loss": train_losses,
            "val_loss": val_losses,
            "train_accuracy": train_accuracies,
            "val_accuracy": val_accuracies,
            "learning_rate": learning_rates
        },
        "best_epoch": best_epoch,
        "best_val_loss": best_val_loss,
        "params": best_params
    }


# ================================================
# STEP 4: RUN THE THREE EXPERIMENTS
# ================================================

models = [
    train_mlp(use_batchnorm=False, use_schedule=False),
    train_mlp(use_batchnorm=True, use_schedule=False),
    train_mlp(use_batchnorm=True, use_schedule=True)
]

for model in models:
    print(f"\n{model['name']}")
    print("Best epoch:", model["best_epoch"])
    print("Best validation loss:", round(model["best_val_loss"], 4))


# ================================================
# STEP 5: TEST THE BEST CHECKPOINT OF EACH MODEL
# ================================================

print("\n--- Test Results ---")

for model in models:
    p = model["params"]
    Z = X_test @ p["W1"] + p["b1"]

    if model["use_batchnorm"]:
        Z = (
            (Z - p["running_mean"])
            / np.sqrt(p["running_var"] + 1e-5)
        )
        Z = p["gamma"] * Z + p["beta"]

    A = relu(Z)
    probabilities = softmax(A @ p["W2"] + p["b2"])
    predictions = np.argmax(probabilities, axis=1)

    print(
        f"{model['name']}: "
```

```
        f"{accuracy_score(y_test, predictions):.4f}"
    )


# ================================================
# STEP 6: PLOT LOSS CURVES
# ================================================

plt.figure(figsize=(9, 5))

for model in models:
    plt.plot(
        model["history"]["val_loss"],
        label=model["name"]
    )

plt.xlabel("Epoch")
plt.ylabel("Validation cross-entropy loss")
plt.title("Batch Normalization and Learning-Rate Scheduling")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("18_validation_loss.png", dpi=200)
plt.show()


# ================================================
# STEP 7: PLOT THE LEARNING-RATE SCHEDULE
# ================================================

plt.figure(figsize=(8, 4))

plt.plot(
    models[2]["history"]["learning_rate"]
)

plt.xlabel("Epoch")
plt.ylabel("Learning rate")
plt.title("Step-Decay Learning-Rate Schedule")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("18_learning_rate.png", dpi=200)
plt.show()
```

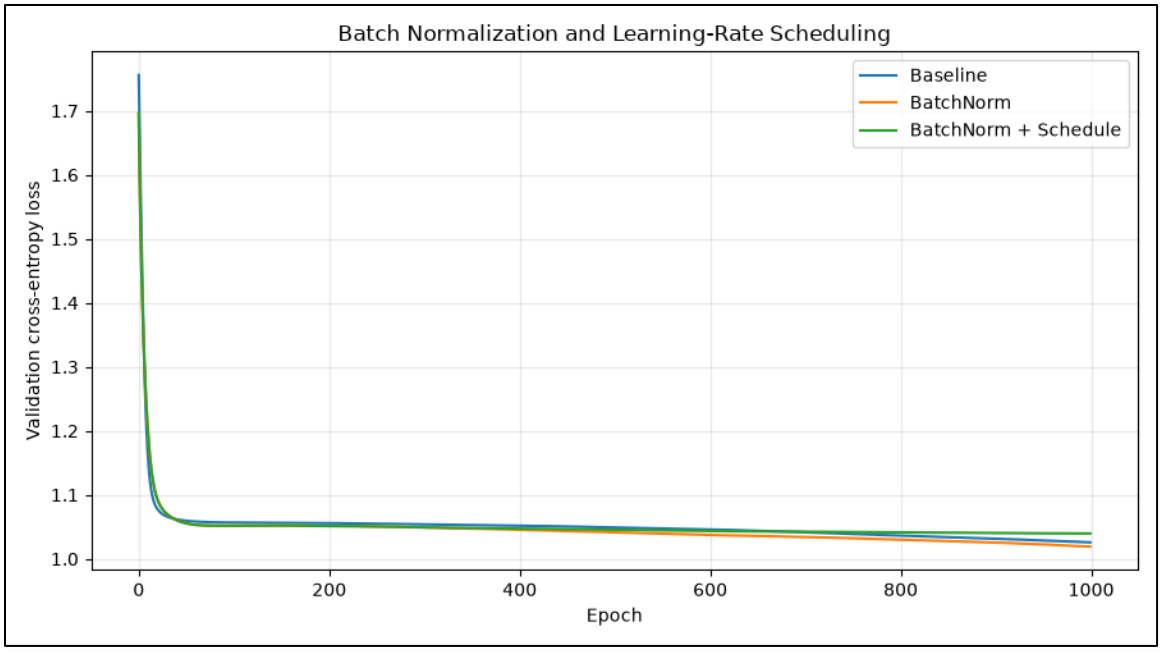
```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 18_batchnorm_lr_schedule.py

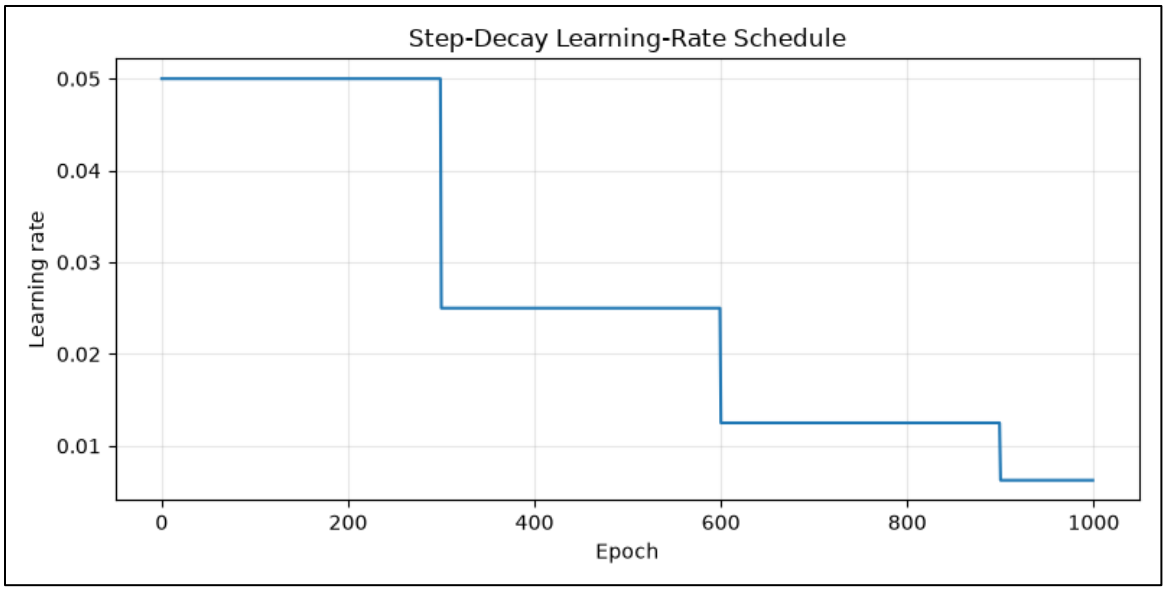
Baseline
Best epoch: 1000
Best validation loss: 1.0258

BatchNorm
Best epoch: 1000
Best validation loss: 1.0192

BatchNorm + Schedule
Best epoch: 1000
Best validation loss: 1.0396

--- Test Results ---
Baseline: 0.4444
BatchNorm: 0.4278
BatchNorm + Schedule: 0.4278
```





Optimizers: SGD, Momentum, and Adam-

A. **SGD:** SGD uses the current gradient to update a parameter: $\theta_{t+1} = \theta_t - \eta g_t$
For example, suppose a weight is 2.0, its gradient is 0.5, and the learning rate is 0.1, then

$$\theta_{\text{new}} = 2.0 - (0.1)(0.5) = 1.95$$

The weight decreases from 2.0 to 1.95.

B. **Momentum:** Momentum maintains a running velocity:

$$v_t = \beta v_{t-1} + g_t$$
$$\theta_{t+1} = \theta_t - \eta v_t$$

Here, β controls how much of the previous velocity is retained. A common value is 0.9.
Momentum builds up when gradients consistently point in the same direction, while reducing some fluctuations in the update direction.

C. Adam

Adam maintains two running estimates:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

It then corrects their initialization bias:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$
$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

Finally, it updates the parameter:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Typical starting values are $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$.

The important difference is that Adam uses both a moving average of gradients and a moving average of squared gradients to adapt the update size for each parameter.

19 optimizers.py

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score


# ================================================
# STEP 1: GENERATE THREE-CLASS SPIRAL DATA
# ================================================

def make_three_spirals(samples_per_class=300, noise=0.08, seed=42):
    rng = np.random.default_rng(seed)
    X_parts, y_parts = [], []

    for class_id in range(3):
        t = np.linspace(0, 1, samples_per_class)
        radius = 0.2 + 1.8 * t
        angle = 2 * np.pi * 1.25 * t + 2 * np.pi * class_id / 3

        points = np.column_stack((
            radius * np.cos(angle),
            radius * np.sin(angle)
        ))
        points += rng.normal(0, noise, points.shape)

        X_parts.append(points)
        y_parts.append(np.full(samples_per_class, class_id))

    X = np.vstack(X_parts)
    y = np.concatenate(y_parts)

    order = rng.permutation(len(y))
    return X[order], y[order]


X, y = make_three_spirals()

X_train_all, X_test, y_train_all, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

X_train, X_val, y_train, y_val = train_test_split(
    X_train_all, y_train_all,
    test_size=0.20, random_state=21, stratify=y_train_all
)
```

```
# Standardize using training data only.
mean = X_train.mean(axis=0)
std = X_train.std(axis=0)

X_train = (X_train - mean) / std
X_val = (X_val - mean) / std
X_test = (X_test - mean) / std

Y_train = np.eye(3)[y_train]
Y_val = np.eye(3)[y_val]


# ================================================
# STEP 2: ACTIVATION, SOFTMAX, AND LOSS
# ================================================

def relu(z):
    return np.maximum(0, z)


def softmax(z):
    z = z - np.max(z, axis=1, keepdims=True)
    e = np.exp(z)
    return e / np.sum(e, axis=1, keepdims=True)


def cross_entropy(p, one_hot_labels):
    eps = 1e-12
    correct_p = np.sum(p * one_hot_labels, axis=1)
    return -np.mean(np.log(correct_p + eps))


# ================================================
# STEP 3: INITIALIZE THE NETWORK
# ================================================

def initialize_network(seed=1, hidden_size=32):
    rng = np.random.default_rng(seed)

    W1 = rng.normal(
        0, np.sqrt(2 / 2), (2, hidden_size)
    )
    b1 = np.zeros((1, hidden_size))

    W2 = rng.normal(
        0, np.sqrt(2 / hidden_size), (hidden_size, 3)
    )
    b2 = np.zeros((1, 3))

    return {
        "W1": W1,
        "b1": b1,
        "W2": W2,
        "b2": b2
    }


def forward(X, params):
    Z1 = X @ params["W1"] + params["b1"]
    A1 = relu(Z1)

    Z2 = A1 @ params["W2"] + params["b2"]
    P = softmax(Z2)
```

```
    cache = (Z1, A1)
    return P, cache


# ================================================
# STEP 4: CALCULATE GRADIENTS
# ================================================

def calculate_gradients(X, Y, params):
    m = len(X)

    P, (Z1, A1) = forward(X, params)

    dZ2 = (P - Y) / m

    dW2 = A1.T @ dZ2
    db2 = np.sum(dZ2, axis=0, keepdims=True)

    dA1 = dZ2 @ params["W2"].T
    dZ1 = dA1 * (Z1 > 0)

    dW1 = X.T @ dZ1
    db1 = np.sum(dZ1, axis=0, keepdims=True)

    gradients = {
        "W1": dW1,
        "b1": db1,
        "W2": dW2,
        "b2": db2
    }

    return gradients


# ================================================
# STEP 5: DEFINE THE OPTIMIZERS
# ================================================

def optimizer_update(
    params,
    gradients,
    state,
    optimizer_name,
    learning_rate,
    step
):
    beta = 0.9
    beta1 = 0.9
    beta2 = 0.999
    epsilon = 1e-8

    for key in params:
        g = gradients[key]

        if optimizer_name == "SGD":
            params[key] -= learning_rate * g

        elif optimizer_name == "Momentum":
            state[key] = beta * state[key] + g
            params[key] -= learning_rate * state[key]

        elif optimizer_name == "Adam":
```

```
            state["m"][key] = (
                beta1 * state["m"][key]
                + (1 - beta1) * g
            )

            state["v"][key] = (
                beta2 * state["v"][key]
                + (1 - beta2) * (g ** 2)
            )

            m_hat = state["m"][key] / (1 - beta1 ** step)
            v_hat = state["v"][key] / (1 - beta2 ** step)

            params[key] -= (
                learning_rate * m_hat
                / (np.sqrt(v_hat) + epsilon)
            )

        else:
            raise ValueError("Unknown optimizer")


# ================================================
# STEP 6: TRAIN A MODEL
# ================================================

def train_model(optimizer_name, epochs=1500):
    # Identical starting parameters for each optimizer.
    params = initialize_network(seed=1)

    if optimizer_name == "SGD":
        learning_rate = 0.1
    elif optimizer_name == "Momentum":
        learning_rate = 0.05
    else:
        learning_rate = 0.01

    # Initialize optimizer-specific state.
    if optimizer_name in ("SGD", "Momentum"):
        state = {
            key: np.zeros_like(value)
            for key, value in params.items()
        }
    else:
        state = {
            "m": {
                key: np.zeros_like(value)
                for key, value in params.items()
            },
            "v": {
                key: np.zeros_like(value)
                for key, value in params.items()
            }
        }

    history = {
        "train_loss": [],
        "val_loss": [],
        "val_accuracy": []
    }

    best_val_loss = float("inf")
    best_params = None
```

```
    best_epoch = 0

    for epoch in range(epochs):
        # Forward pass and gradients.
        gradients = calculate_gradients(
            X_train, Y_train, params
        )

        # Update parameters.
        optimizer_update(
            params,
            gradients,
            state,
            optimizer_name,
            learning_rate,
            epoch + 1
        )

        # Evaluate updated model.
        train_p, _ = forward(X_train, params)
        val_p, _ = forward(X_val, params)

        train_loss = cross_entropy(train_p, Y_train)
        val_loss = cross_entropy(val_p, Y_val)

        val_pred = np.argmax(val_p, axis=1)
        val_accuracy = accuracy_score(y_val, val_pred)

        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_loss)
        history["val_accuracy"].append(val_accuracy)

        if val_loss < best_val_loss:
            best_val_loss = val_loss
            best_epoch = epoch + 1
            best_params = {
                key: value.copy()
                for key, value in params.items()
            }

    print(f"\n{optimizer_name}")
    print("Best epoch:", best_epoch)
    print("Best validation loss:", round(best_val_loss, 4))
    print(
        "Best validation accuracy:",
        round(max(history["val_accuracy"]), 4)
    )

    return {
        "name": optimizer_name,
        "history": history,
        "params": best_params,
        "best_epoch": best_epoch,
        "best_val_loss": best_val_loss
    }


# ================================================
# STEP 7: COMPARE THE THREE OPTIMIZERS
# ================================================

models = [
    train_model("SGD"),
```

```
    train_model("Momentum"),
    train_model("Adam")
]


# ================================================
# STEP 8: FINAL TEST EVALUATION
# ================================================

print("\n--- Test Accuracy ---")

for model in models:
    probabilities, _ = forward(X_test, model["params"])
    predictions = np.argmax(probabilities, axis=1)

    accuracy = accuracy_score(y_test, predictions)

    print(f"{model['name']}: {accuracy:.4f}")


# ================================================
# STEP 9: PLOT TRAINING LOSS
# ================================================

plt.figure(figsize=(9, 5))

for model in models:
    plt.plot(
        model["history"]["train_loss"],
        label=model["name"]
    )

plt.xlabel("Epoch")
plt.ylabel("Training cross-entropy loss")
plt.title("Optimizer Comparison: Training Loss")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("19_training_loss.png", dpi=200)
plt.show()


# ================================================
# STEP 10: PLOT VALIDATION LOSS
# ================================================

plt.figure(figsize=(9, 5))

for model in models:
    plt.plot(
        model["history"]["val_loss"],
        label=model["name"]
    )

plt.xlabel("Epoch")
plt.ylabel("Validation cross-entropy loss")
plt.title("Optimizer Comparison: Validation Loss")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("19_validation_loss.png", dpi=200)
plt.show()
```

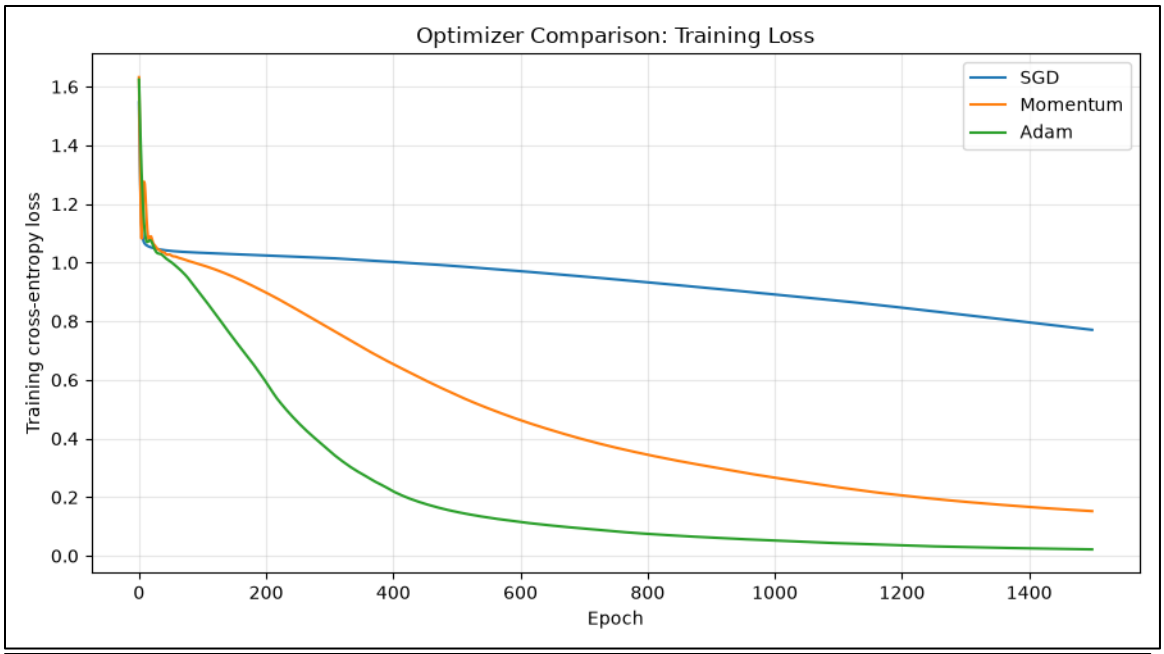
```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 19_optimizers.py

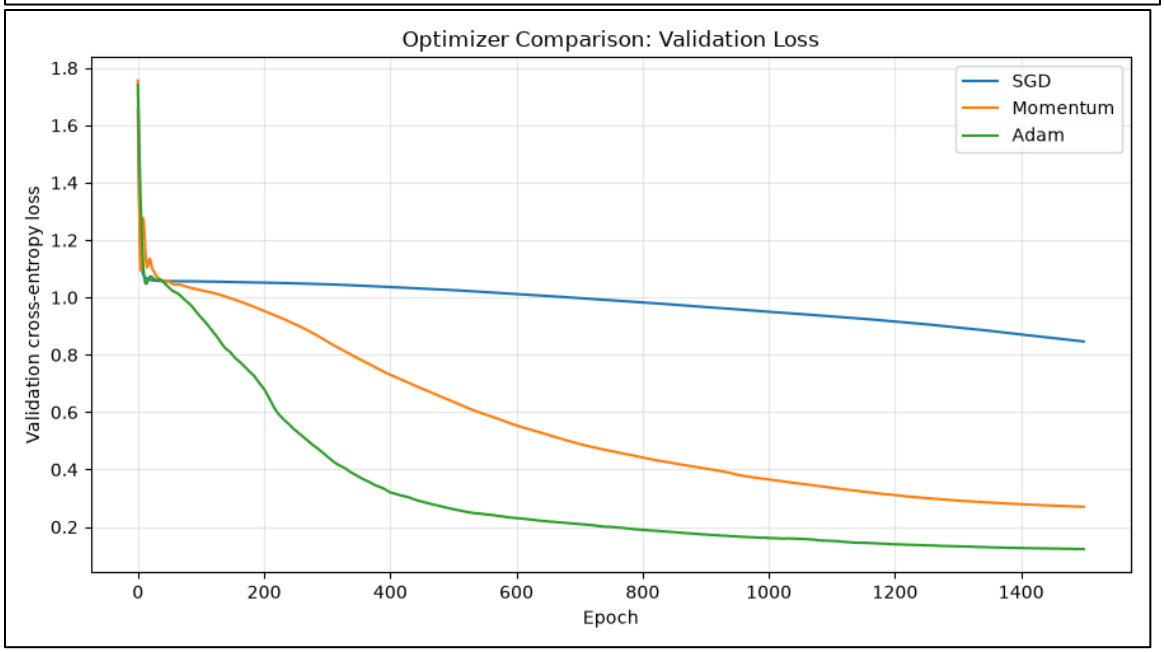
SGD
Best epoch: 1500
Best validation loss: 0.8465
Best validation accuracy: 0.5625

Momentum
Best epoch: 1500
Best validation loss: 0.2702
Best validation accuracy: 0.9306

Adam
Best epoch: 1500
Best validation loss: 0.1225
Best validation accuracy: 0.9653

--- Test Accuracy ---
SGD: 0.6056
Momentum: 0.9667
Adam: 0.9889
```





Mini Deep-Learning Framework-

20_mini_deep_learning_framework.py

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report


# ============================================================
# PART 1: DATASET
# ============================================================

def make_three_spirals(samples_per_class=300, noise=0.08, seed=42):
    rng = np.random.default_rng(seed)
    X_parts, y_parts = [], []

    for class_id in range(3):
        t = np.linspace(0, 1, samples_per_class)
        radius = 0.2 + 1.8 * t
        angle = 2 * np.pi * 1.25 * t + 2 * np.pi * class_id / 3

        points = np.column_stack([
            radius * np.cos(angle),
            radius * np.sin(angle)
        ])

        points += rng.normal(0, noise, points.shape)

        X_parts.append(points)
        y_parts.append(np.full(samples_per_class, class_id))

    X = np.vstack(X_parts)
    y = np.concatenate(y_parts)

    order = rng.permutation(len(y))
    return X[order], y[order]


X, y = make_three_spirals()

# 64% train, 16% validation, 20% test.
X_train_all, X_test, y_train_all, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

X_train, X_val, y_train, y_val = train_test_split(
    X_train_all,
    y_train_all,
    test_size=0.20,
    random_state=21,
    stratify=y_train_all
)

# Standardize using training data only.
mean = X_train.mean(axis=0)
std = X_train.std(axis=0)

X_train = (X_train - mean) / std
X_val = (X_val - mean) / std
X_test = (X_test - mean) / std


# ============================================================
# PART 2: LAYERS
# ============================================================
```

```
class Dense:
    """Fully connected layer: output = input @ W + b."""

    def __init__(self, input_size, output_size, seed=1):
        rng = np.random.default_rng(seed)

        # He-style initialization for a layer followed by ReLU.
        self.W = rng.normal(
            0,
            np.sqrt(2 / input_size),
            size=(input_size, output_size)
        )
        self.b = np.zeros((1, output_size))

        self.dW = np.zeros_like(self.W)
        self.db = np.zeros_like(self.b)
        self.input = None

    def forward(self, X):
        self.input = X
        return X @ self.W + self.b

    def backward(self, grad_output):
        # grad_output is dLoss/dOutput.
        n = self.input.shape[0]

        self.dW = self.input.T @ grad_output
        self.db = np.sum(grad_output, axis=0, keepdims=True)

        # Gradient passed to the previous layer.
        grad_input = grad_output @ self.W.T

        return grad_input

    def parameters(self):
        return [
            (self.W, self.dW),
            (self.b, self.db)
        ]


class ReLU:
    """Rectified Linear Unit activation."""

    def __init__(self):
        self.input = None

    def forward(self, X):
        self.input = X
        return np.maximum(0, X)

    def backward(self, grad_output):
        return grad_output * (self.input > 0)


# ============================================================
# PART 3: NEURAL NETWORK
# ============================================================

class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size):
        self.layers = [
            Dense(input_size, hidden_size, seed=1),
            ReLU(),
            Dense(hidden_size, output_size, seed=2)
        ]

    def forward(self, X):
        output = X
```

```
        for layer in self.layers:
            output = layer.forward(output)

        # Return logits, not probabilities.
        return output

    def backward(self, grad_output):
        for layer in reversed(self.layers):
            grad_output = layer.backward(grad_output)

    def parameters(self):
        result = []

        for layer in self.layers:
            if isinstance(layer, Dense):
                result.extend(layer.parameters())

        return result


# ============================================================
# PART 4: SOFTMAX AND CROSS-ENTROPY
# ============================================================

def softmax(logits):
    # Subtract maximum to improve numerical stability.
    shifted = logits - np.max(logits, axis=1, keepdims=True)
    exp_values = np.exp(shifted)

    return exp_values / np.sum(
        exp_values, axis=1, keepdims=True
    )


def cross_entropy_and_gradient(logits, labels):
    n = len(labels)
    probabilities = softmax(logits)

    # Mean cross-entropy.
    correct_probabilities = probabilities[
        np.arange(n), labels
    ]

    loss = -np.mean(
        np.log(np.clip(correct_probabilities, 1e-12, 1.0))
    )

    # Gradient of mean softmax cross-entropy with respect to logits.
    grad = probabilities.copy()
    grad[np.arange(n), labels] -= 1
    grad /= n

    return loss, grad, probabilities


# ============================================================
# PART 5: REUSABLE OPTIMIZER
# ============================================================

class Optimizer:
    def __init__(
        self,
        name="adam",
        learning_rate=0.01,
        beta1=0.9,
        beta2=0.999,
        epsilon=1e-8,
        momentum=0.9
    ):
        self.name = name.lower()
```

```
        self.lr = learning_rate
        self.beta1 = beta1
        self.beta2 = beta2
        self.epsilon = epsilon
        self.momentum = momentum

        self.step = 0
        self.state = {}

        if self.name not in ("sgd", "momentum", "adam"):
            raise ValueError("Choose sgd, momentum, or adam.")

    def update(self, parameters):
        self.step += 1

        for index, (param, grad) in enumerate(parameters):
            if self.name == "sgd":
                param -= self.lr * grad

            elif self.name == "momentum":
                if index not in self.state:
                    self.state[index] = np.zeros_like(param)

                velocity = self.state[index]
                velocity *= self.momentum
                velocity += grad

                param -= self.lr * velocity

            elif self.name == "adam":
                if index not in self.state:
                    self.state[index] = {
                        "m": np.zeros_like(param),
                        "v": np.zeros_like(param)
                    }

                state = self.state[index]
                state["m"] = (
                    self.beta1 * state["m"]
                    + (1 - self.beta1) * grad
                )
                state["v"] = (
                    self.beta2 * state["v"]
                    + (1 - self.beta2) * grad**2
                )

                m_hat = state["m"] / (
                    1 - self.beta1**self.step
                )
                v_hat = state["v"] / (
                    1 - self.beta2**self.step
                )

                param -= (
                    self.lr * m_hat
                    / (np.sqrt(v_hat) + self.epsilon)
                )


# ============================================================
# PART 6: TRAINING AND EVALUATION FUNCTIONS
# ============================================================

def predict(network, X):
    logits = network.forward(X)
    return np.argmax(logits, axis=1)


def evaluate(network, X, y):
    logits = network.forward(X)
```

```
    loss, _, probabilities = cross_entropy_and_gradient(
        logits, y
    )

    predictions = np.argmax(probabilities, axis=1)
    accuracy = accuracy_score(y, predictions)

    return loss, accuracy


def train(
    network,
    optimizer,
    X_train,
    y_train,
    X_val,
    y_val,
    epochs=1500
):
    history = {
        "train_loss": [],
        "val_loss": [],
        "train_accuracy": [],
        "val_accuracy": []
    }

    best_val_loss = float("inf")
    best_epoch = 0
    best_parameters = None

    for epoch in range(epochs):

        # Forward pass.
        logits = network.forward(X_train)

        # Loss and output gradient.
        loss, grad, _ = cross_entropy_and_gradient(
            logits, y_train
        )

        # Backpropagate through all layers.
        network.backward(grad)

        # Update weights and biases.
        optimizer.update(network.parameters())

        # Evaluate after updating the parameters.
        train_loss, train_accuracy = evaluate(
            network, X_train, y_train
        )
        val_loss, val_accuracy = evaluate(
            network, X_val, y_val
        )

        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_loss)
        history["train_accuracy"].append(train_accuracy)
        history["val_accuracy"].append(val_accuracy)

        # Save the best validation-loss checkpoint.
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            best_epoch = epoch + 1

            best_parameters = [
                (layer.W.copy(), layer.b.copy())
                for layer in network.layers
                if isinstance(layer, Dense)
            ]
```

```
        if (epoch + 1) % 250 == 0:
            print(
                f"Epoch {epoch + 1:4d} | "
                f"Train loss: {train_loss:.4f} | "
                f"Validation loss: {val_loss:.4f} | "
                f"Validation accuracy: {val_accuracy:.4f}"
            )

    # Restore the best checkpoint.
    dense_layers = [
        layer for layer in network.layers
        if isinstance(layer, Dense)
    ]

    for layer, (W, b) in zip(dense_layers, best_parameters):
        layer.W = W
        layer.b = b

    print("\nBest validation epoch:", best_epoch)
    print("Best validation loss:", round(best_val_loss, 4))

    return history


# ============================================================
# PART 7: BUILD AND TRAIN THE FINAL NETWORK
# ============================================================

network = NeuralNetwork(
    input_size=2,
    hidden_size=32,
    output_size=3
)

optimizer = Optimizer(
    name="adam",
    learning_rate=0.01
)

history = train(
    network,
    optimizer,
    X_train,
    y_train,
    X_val,
    y_val,
    epochs=1500
)


# ============================================================
# PART 8: FINAL TEST EVALUATION
# ============================================================

test_loss, test_accuracy = evaluate(
    network, X_test, y_test
)

print("\n========== FINAL TEST RESULTS ==========")
print("Test loss:", round(test_loss, 4))
print("Test accuracy:", round(test_accuracy, 4))

test_predictions = predict(network, X_test)

print("\nClassification report:")
print(
    classification_report(
        y_test,
        test_predictions,
        zero_division=0
```

```
    )
)


# ============================================================
# PART 9: PLOT TRAINING AND VALIDATION LOSS
# ============================================================

plt.figure(figsize=(8, 5))

plt.plot(history["train_loss"], label="Training loss")
plt.plot(history["val_loss"], label="Validation loss")

plt.xlabel("Epoch")
plt.ylabel("Cross-entropy loss")
plt.title("Mini Deep-Learning Framework")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("20_training_loss.png", dpi=200)
plt.show()


# ============================================================
# PART 10: VISUALIZE DECISION BOUNDARIES
# ============================================================

x_min, x_max = X[:, 0].min() - 0.3, X[:, 0].max() + 0.3
y_min, y_max = X[:, 1].min() - 0.3, X[:, 1].max() + 0.3

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 250),
    np.linspace(y_min, y_max, 250)
)

grid = np.column_stack((xx.ravel(), yy.ravel()))
grid_scaled = (grid - mean) / std

grid_predictions = predict(network, grid_scaled)
regions = grid_predictions.reshape(xx.shape)

plt.figure(figsize=(8, 6))

plt.contourf(
    xx,
    yy,
    regions,
    levels=[-0.5, 0.5, 1.5, 2.5],
    alpha=0.3
)

for class_id in range(3):
    plt.scatter(
        X_test[y_test == class_id, 0],
        X_test[y_test == class_id, 1],
        label=f"Class {class_id}",
        s=20
    )

plt.title("Final MLP Decision Boundaries")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.grid(True, alpha=0.2)
plt.tight_layout()
plt.savefig("20_decision_boundaries.png", dpi=200)
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\02_multiclass_classification> py 20_mini_deep_learning_framework.py
Epoch  250 | Train loss: 0.5319 | Validation loss: 0.6218 | Validation accuracy: 0.7500
Epoch  500 | Train loss: 0.1699 | Validation loss: 0.3084 | Validation accuracy: 0.9444
Epoch  750 | Train loss: 0.0752 | Validation loss: 0.2504 | Validation accuracy: 0.9722
Epoch 1000 | Train loss: 0.0448 | Validation loss: 0.2546 | Validation accuracy: 0.9722
Epoch 1250 | Train loss: 0.0307 | Validation loss: 0.2676 | Validation accuracy: 0.9722
Epoch 1500 | Train loss: 0.0224 | Validation loss: 0.2676 | Validation accuracy: 0.9722

Best validation epoch: 844
Best validation loss: 0.2464

========== FINAL TEST RESULTS ==========
Test loss: 0.1648
Test accuracy: 0.9722

Classification report:
              precision    recall  f1-score   support

           0       0.98      0.98      0.98        60
           1       0.98      0.95      0.97        60
           2       0.95      0.98      0.97        60

    accuracy                           0.97       180
   macro avg       0.97      0.97      0.97       180
weighted avg       0.97      0.97      0.97       180
```

